

POS ターミナル
RZ-A476S/S396S
POS Foundation Object 説明書

2024 年 2 月 5 日 第 1.0 版

シャープ株式会社
スマートビジネスソリューション事業本部
ワークプレイスイノベーション事業部
システム第一開発部

はじめに

本書では、POS ターミナル RZ-A476S/A396S にてシャープ(株)より提供する POS Foundation Object (以降 PFO と記載する)の説明をしています。

PFO とは、POS アプリケーション構築を支援するためのオブジェクト(ソフト部品)環境で、画面系、ロジック系、システム制御系、デバイス制御系の API で構成されています。

PFO は、ActiveX コントロール(OCX や OLE コントロールとも呼ばれる)や ActiveX サーバー(DLL または EXE 型のオートメーションオブジェクト)、DLL プロシージャ(Pascal 形式)の形態で提供されます。

本 PFO は、アプリケーションプログラム開発システムとして、Visual C# (以下 VC#と記載)、Visual Basic (以下 VB.NET と記載)に対応しています。

本書は、アプリケーション開発システム(Visual Studio 2019、.NET Framework、Active コンポーネント(COM)、Visual Basic 等)に精通している事を前提として記述しています。

**注意 : RZ-A476S/A396S は、58mm 幅プリンタと 80mm 幅プリンタの 2 種類から選択可能です。
特に断らない場合は 58mm 幅を代表として説明を行っています。**

※ OPOS は Open POS の略です。

※ Microsoft、Windows、Visual Studio、Visual Basic、Visual C#、ActiveX、Active Directory、IntelliMirror、DirectX は米国 Microsoft Corporation の米国およびその他の国における登録商標または商標です。

※ Windows の正式名称は Microsoft Windows Operating System です。

※ COM は Common Object Model、DCOM は Distributed Common Object Model の略です。

本書内容の一部または全部の複製、変更、転載、電子装置による読み取りを禁じます。

本書の著作権は、シャープ株式会社にあります。

本書は改良のために事前に予告なく内容の一部が変更されることがあります。

改訂履歴

(2023 年 10 月 10 日 第 1.0 版)

新規作成

(2024 年 2 月 5 日)

下記を修正。

全体のフッターからタイトルを削除

付録 F.を削除(PFO の機能ではない)

目次

改訂履歴	3
第 1 章 概要.....	7
1.1. POS Foundation Object	7
1.2. PFO の利用方法	8
1.2.1. ミドルウェア別利用方法 (.NET)	8
1.2.2. 特記事項 (.NET)	9
第 2 章 PFO 一覧	10
2.1. 構成.....	10
2.2. 機能要約	11
第 3 章 編集.....	14
3.1. イメージ編集(ActiveX Control)	14
3.1.1. 概要	14
3.1.2. オブジェクト形態	14
3.1.3. 機能	14
3.1.4. 共通 I/F サマリー	14
3.1.5. プロパティ詳細	15
3.1.6. メソッド詳細	16
3.1.7. イメージ編集シーケンス	18
3.1.8. コントロールの表示と座標系	18
3.1.9. 補足	18
第 4 章 演算/変換	19
4.1. 概要.....	19
4.2. モジュール形態.....	19
4.3. 演算・変換 (.NET - ActiveX EXE)	20
4.3.1. 共通 I/F サマリー	20
4.3.2. メソッド詳細	20
4.3.3. 文字コード変換.....	26
第 5 章 電子ジャーナル.....	28
5.1. 電子ジャーナル(ActiveX Control)	28
5.1.1. 概要	28
5.1.2. オブジェクト形態	31
5.1.3. 共通 I/F サマリー	31
5.1.4. プロパティ詳細	32
5.1.5. メソッド詳細	34
5.1.6. 設定ファイル (PFOEJCODE.ini)	40
5.1.7. 特記事項.....	40
5.2. .NET 版 電子ジャーナルビューアー(.NET EXE)	41
5.2.1. 概要	41
5.2.2. モジュール形態	43
5.2.3. UI 機能詳細.....	43
5.2.4. 設定ファイル(EJViewer.ini).....	48
5.2.5. 特記事項.....	50

5.3. 電子ジャーナルファイルチェック (ActiveX DLL)	51
5.3.1. 概要	51
5.3.2. オブジェクト形態	51
5.3.3. 共通 I/F サマリー	51
5.3.4. メソッド詳細	52
第 6 章 デバイス支援	53
6.1. 液晶制御 (ActiveX DLL)	53
6.1.1. 概要	53
6.1.2. オブジェクト形態	53
6.1.3. 共通 I/F サマリー	53
6.1.4. メソッド詳細	53
6.1.5. 特記事項	53
6.2. POS プリンタ外字 (DLL)	55
6.2.1. 概要	55
6.2.2. 共通 I/F サマリー	55
6.2.3. 関数詳細	55
6.3. POS キーボード制御 (DLL)	56
6.3.1. 概要	56
6.3.2. 共通 I/F サマリー	56
6.3.3. 関数詳細	56
第 7 章 システム支援	58
7.1. システム起動 (DLL)	58
7.1.1. 概要	58
7.1.2. 共通 I/F サマリー	58
7.1.3. 関数詳細	58
7.2. システム停止 (DLL)	59
7.2.1. 概要	59
7.2.2. 共通 I/F サマリー	59
7.2.3. 関数詳細	59
7.3. ミラーリング監視 (DLL)	60
7.3.1. 概要	60
7.3.2. 共通 I/F サマリー	60
7.3.3. 関数詳細	60
第 8 章 保守監視	63
8.1. SMART 情報	63
8.1.1. 概要	63
8.1.2. 共通 I/F サマリー	63
8.1.3. 関数詳細	63
8.2. 内蔵バッテリー情報 (DLL)	69
8.2.1. 概要	69
8.2.2. 共通 I/F サマリー	69
8.2.3. 関数詳細	69
8.3. POS ハードウェア情報 (DLL)	71
8.3.1. 概要	71
8.3.2. 共通 I/F サマリー	71
8.3.3. 関数詳細	71

第 9 章 プロセス情報	73
9.1. プロセス情報取得 (ActiveX Control)	73
9.1.1. 概要	73
9.1.2. オブジェクト形態	73
9.1.3. 機能	73
9.1.4. 共通 I/F サマリー	73
9.1.5. メソッド詳細	73
9.1.6. ログファイル詳細	74
9.1.7. 設定ファイル (POSProc.ini)	75
第 10 章 操作ログ	76
10.1. 操作ログ (ActiveX Control)	76
10.1.1. 概要	76
10.1.2. モジュール形態	77
10.1.3. 操作ログ対応モジュールの利用方法	77
10.1.4. GUI 操作ログ対象オブジェクト	77
10.1.5. 設定ファイル (posoplog.ini)	78
10.1.6. ログデータフォーマット	79
10.1.7. 操作情報、補足フォーマット	80
10.1.8. 操作ログ制約・注意事項	86
付録 A. PFO 定数一覧	87
付録 B. Calcudgt 計算規則 (方法)	89
付録 C. 演算データ形式	90
付録 D. 演算変換テーブル	91
付録 E. ブザー制御	93

第1章 概要

1.1. POS Foundation Object

POS ビジネスオブジェクトとは、ビジネスオブジェクトアーキテクチャ(業務アプリケーションの部品化)をベースに構築したシャープ仕様のオブジェクト指向プログラミング環境で、POS アプリケーションの生産性を向上する目的のものです。

RZ-A476S/A396S では、POS 汎用オブジェクト層に属した PFO (POS Foundation Object)を提供し、業務で利用する機能単位部品や、業種・業務に特化した部品開発を支援します。

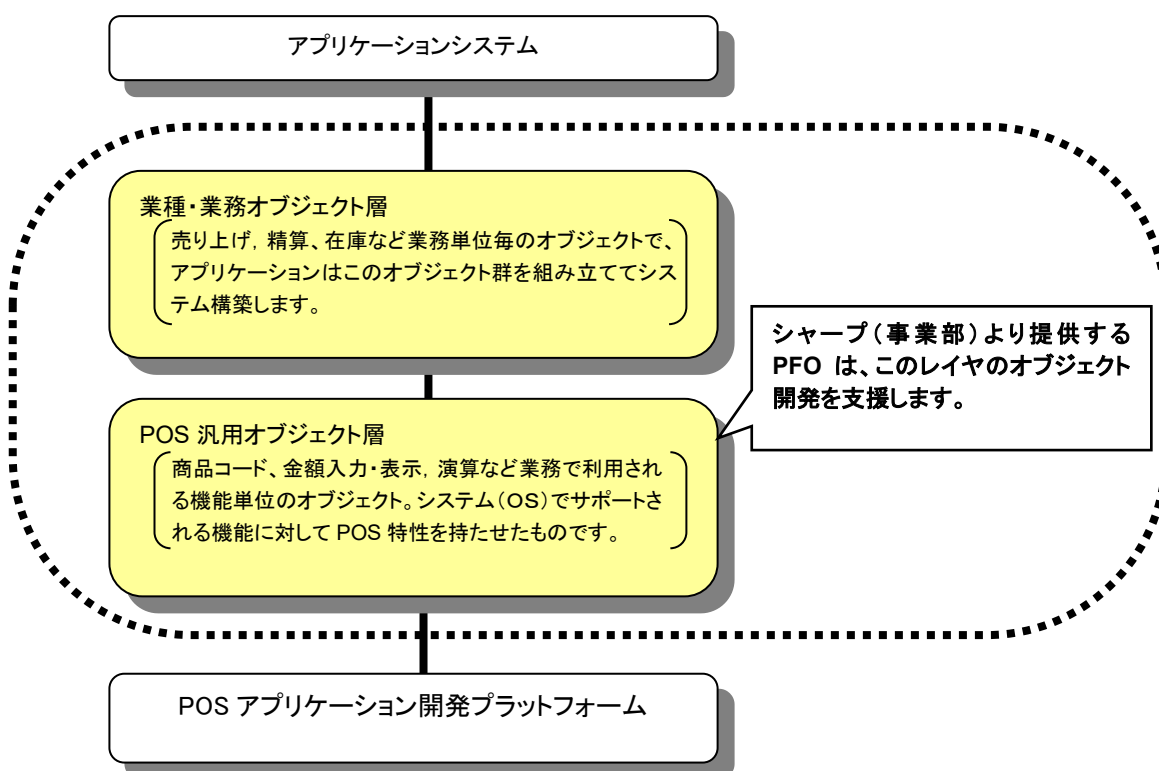
POS ビジネスオブジェクトはオブジェクト(部品化)技術として、Windows のオブジェクト技術(ActiveX, 旧名 OLE)を採用しています。なお、一部の機能は DLL プロシージャ(Pascal 形式)として提供しています。

なお、POS ビジネスオブジェクトは 32bit 用の DLL プロシージャのみ提供します。

PFO 特徴

1. POS でサポートされる差別化機能の利用
2. 複雑化傾向にあるソフト技術の吸収効果

PFO 概念図



PFO (POS Foundation Object) について

PFO (POS Foundation Object) は、POS 汎用オブジェクト層に位置したオブジェクト(部品)で、画面系、ロジック系、システム制御系および POS デバイス制御系の部品から構成され、アプリケーションより自由に組み合わせて利用することが可能な POS 汎用オブジェクト群です。

1.2. PFO の利用方法

1.2.1. ミドルウェア別利用方法 (.NET)

VC#、VB.NET から利用する PFO は、ActiveX Control (OCX)、ActiveX DLL (インプロセスオートメーションオブジェクト)、ActiveX EXE (アウトプロセスオートメーションオブジェクト)、DLL (Pascal 形式)、.NET Class Library (DLL)、.NET アプリケーションのいずれかの形態で提供されます。それぞれのミドルウェアは、以下の手順で利用します。

【ActiveX Control (OCX) 形態の API 利用方法】

1. オブジェクトの参照設定

「ツール(T) | ツールボックス アイテムの選択(X)」メニュー、もしくはデザインモードのツールボックスから「右クリック | アイテムの選択(I)」を選択します
ツールボックス アイテムの選択ダイアログの **COM** コンポーネントタブの一覧で、利用するオブジェクトの識別子にチェックをします。

2. プログラム実装方法

ツールボックスより目的のコントロールを選択し、フォームに貼り付けます。

API の呼び出しは以下のように記述します。

```
result = POSOBJ.XXX(xxx)
```

※ **ActiveX Control** の参照設定を行うとラッパーライブラリが作成されます。
これらのラッパーライブラリは動作時に必要になるのでご注意ください。

【ActiveX DLL/EXE 形態の API 利用方法】

1. オブジェクトの参照設定

「プロジェクト(P) | 参照の追加(R)」メニューを選択します。

参照の追加ダイアログの **COM** タブ一覧で、利用するオブジェクトの識別子にチェックをします。

参照設定を行わないと、PFO で定義されている定数を利用することが出来ません。

2. コード実装方法

オブジェクト変数確保ならびにオブジェクトをクリエイトします。

方法)

```
【ライブラリ名.オブジェクト】 POSOBJ = new 【ライブラリ名.オブジェクト】();
```

API の呼び出しは以下のように記述します。

```
result = POSOBJ.XXX(xxx)
```

※ **ActiveX DLL/ActiveX EXE** の参照設定を行うとラッパーライブラリが作成されます。
ライブラリ名はこれらのラッパーライブラリ名となります。
ライブラリは動作時に必要になるのでご注意ください。

【DLL 形態の API 利用方法】

DllImport 属性を使用して関数の宣言を行います。

例) POS_EXIT 関数を利用する場合

VC#:

```
[DllImport("POSSYS32.dll")]
```

```
public static extern Int32 POS_EXIT(Int32 flag);
```

VB.NET:

```
<DllImport("POSSYS32.dll")> _
```

```
Private Shared Function POS_EXIT(ByVal flag As Int32) As Int32
```

```
End Function
```

【.NET Class Library 形態の API 利用方法】

1. オブジェクトの参照設定

「ツール(T) | ツールボックス アイテムの選択(X)」メニュー、もしくはデザインモードのツールボックスから「右クリック | アイテムの選択(I)」を選択します

ツールボックス アイテムの選択ダイアログの.NET Framework コンポーネントタブの参照より、次のファイルを選択します。

C:\Program Files (x86)\Common Files\Sharp\POS\PFO\OpeLog\SHARP.POS.PFO.GUIOpeLog\1.0.0.0\SHARP.POS.PFO.GUIOpeLog.dll

2. プログラム実装方法

ツールボックスより目的のコントロールを選択し、フォームに貼り付けます。

【.NET アプリケーション形態のモジュール利用方法】

方法 1) 終了を待ち合わせる場合

```
Process ejViewer = new Process();  
ejViewer.StartInfo.FileName = 【EXE ファイルフルパス】;  
ejViewer.StartInfo.Arguments = 【オプション】;  
ejViewer.Start();  
ejViewer.WaitForExit();
```

方法 2) 終了を待ち合わせない場合

```
Process ejViewer = new Process();  
ejViewer.StartInfo.FileName = 【EXE ファイルフルパス】;  
ejViewer.StartInfo.Arguments = 【オプション】;  
ejViewer.Start();
```

1.2.2. 特記事項 (.NET)

- ActiveX 系オブジェクトのスレッディングモデル
シングルスレッドアパートメントモデルとして実装されています。
- String 型引数へのバイトデータ指定方法
.Net では String に直接 BYTE 型データを格納することができない仕様となっています。したがって、String へ BYTE 型データを格納する場合は、BYTE 型データを 2 バイト単位で Char 型にパックした後、String に格納する必要があります。実際のコード事例は、サンプルプログラムを御確認ください。

第2章 PFO 一覧

2.1. 構成

分 類	機能	ファイル名	オブジェクト形態
ロジック (編集/演算/変換)	イメージ編集	PFOIEdit.OCX	ActiveX Control
	演算/変換	PFOCALCNet.exe	ActiveX EXE
電子ジャーナル	電子ジャーナル	PFOEJ.OCX	ActiveX Control
	電子ジャーナルビューアー	EJViewerNET.EXE	.NET EXE
	電子ジャーナルファイルチェック	PFOEJChk.DLL	ActiveX DLL
デバイス支援	液晶制御	PFODisp.DLL	ActiveX DLL
	POS プリンタ外字	POSUTIL.DLL	DLL
	POS キーボード制御		
システム支援	システム起動	POSSYS32.DLL	DLL
	システム停止		
	ミラーリング監視	POSMIRROR.DLL	DLL
保守監視	SMART 情報	POSMIRROR.DLL	DLL
	内蔵バッテリー	POSSYS32.DLL	
	POS ハード情報		
プロセス情報	プロセス情報取得	PFOProc.OCX	ActiveX Control
操作ログ	POS デバイス操作ログ	Msr.ocx	ActiveX Control
		Scanner.ocx	ActiveX Control
	GUI 操作ログ	SHARP.POS.PFO. GUIOpeLog.dll	.NET DLL

以下に PFO インストーラとの関係を示します。

分 類	機能	インストーラ	開発環境	実行環境
ロジック (編集/演算/変換)	イメージ編集	pfobas_setup.exe	必要	必要
	演算/変換			
電子ジャーナル	電子ジャーナル			
	電子ジャーナルビューアー 電子ジャーナルファイルチェック			
デバイス支援	液晶制御	pfobas_setup.exe	必要	
	POS プリンタ外字	pfosys_setup.exe	不要	
	POS キーボード制御			
システム支援	システム起動	pfosys_setup.exe	不要	
	システム停止			
	ミラーリング監視			
保守監視	SMART 情報	pfosys_setup.exe	必要	
	内蔵バッテリー			
	POS ハード情報			
プロセス情報	プロセス情報取得	pfobas_setup.exe	必要	
操作ログ	POS デバイス操作ログ	opoco_setup.exe(※)	必要	
	GUI 操作ログ	pfoopelog_setup.exe		

(※)POS デバイス操作ログは、従来機種では操作ログ版の OPOS を使用する必要がありましたが、RZ-A476S/A396S は標準 OPOS にて操作ログが行えます。POS デバイスの操作ログを有効とするためには、OPOS レジストリに操作ログキーを指定します。詳細は、POS デバイス API 説明書の OPOS レジストリ章を参照ください。

2.2. 機能要約

【ロジック編集】

機能分類	API	機 能
イメージ編集	LoadBitmap	モノクロビットマップファイルをコントロール内の編集領域に読み込む
	PrintText	ビットマップ画像上の CurrentX,CurrentY 座標に文字列 Text を描画する
	PrintTextAt	ビットマップ画像上の X,Y 座標に文字列 Text を描画する
	SaveBitmap	編集領域のビットマップ画像を回転させ、モノクロビットマップ形式でファイルに保存する

【ロジック変換/演算 (.NET)】

機能分類	API	機 能
演算/変換	Calcudgt	チェックデジットの計算
	PStoInt32	符号有り内部 10 進数(PS 型データ)を Int32 型に変換
	NStoInt32	符号有り外部 10 進数(NS 型データ)を Int32 型に変換
	NtoInt32	符号無し外部 10 進数(N 型データ)を Int32 型に変換
	PStoDecimal	符号有り内部 10 進数(PS 型データ)を Decimal 型に変換
	NStoDecimal	符号有り外部 10 進数(NS 型データ)を Decimal 型に変換
	NtoDecimal	符号無し外部 10 進数(N 型データ)を Decimal 型に変換
	SJISStoJIS	SJIS データを JIS データに変換
	JISStoSJIS	JIS データを SJIS データに変換
	JIS7to8	JIS7 データを JIS8 データに変換
	JIS8to7	JIS8 データを JIS7 データに変換
	JIS8toEBC	JIS8 データを EBCDIC データに変換
	EBCtoJIS8	EBCDIC データを JIS8 データに変換

【電子ジャーナル】

機能分類	API	機 能
電子ジャーナル	EJCreate	ジャーナルファイル生成
	EJOpen	ジャーナルファイルのオープン
	EJClose	ジャーナルファイルのクローズ
	EJDelete	ジャーナルファイルの削除
	EJFileFind	ジャーナルファイルの検索
	WriteData	ジャーナルデータ(テキスト)の書き込み(追記記憶)
	SearchData	任意位置からの検索
	ReadData	ジャーナルデータ取得
	ViewData	ジャーナルデータの表示
	ViewClear	ジャーナル表示データのクリア
	RecordSize	ジャーナルデータのレコードサイズを取得
	RecordCounts	ジャーナルデータのレコード数を取得
電子ジャーナルビューアー	(実行ファイル)	電子ジャーナルビューアー
電子ジャーナル ファイルチェック	EJFileChk	電子ジャーナルファイルの診断・修復

【デバイス支援】

機能分類	API	機 能
液晶制御	SetBackLight	LCD バックライト オン/オフ切り替え
	SetBackLightBrightness	LCD バックライト輝度調整
POS プリンタ外字	GetGaijiData	外字ファイルから外字 1 文字のデータを取得
POS キーボード制御	PosKdown	POS キー配置登録

【システム支援】

機能分類	API	機 能
システム起動	WaitPosapl	POS システムの終了処理
システム停止	POS_EXIT	電源 ON 時に起動プロセスの処理完了を待つ
ミラーリング監視	GetMirrorMode	ミラー/単体モードの取得
	GetNumDisks	接続されているドライブの台数
	GetMirrorStatus	ミラー状態の取得
	GetPrimaryDiskStatus	ドライブ 1 の状態の取得
	GetSecondaryDiskStatus	ドライブ 2 の状態の取得

【保守監視】

機能分類	API	機 能
SMART 情報	GetDiskInfo	ドライブのモデル名とシリアル番号の取得
	GetDiskID	ドライブの ID 情報の取得
	GetSMARTInfo	ドライブの SMART 情報の取得
	GetSMARTAttribute	ドライブの SMART 属性情報の取得
	GetSMARTThreshold	ドライブの SMART 閾値情報の取得
内蔵バッテリー	GetBatteryInfo	バッテリー情報の取得
POS ハード情報	GetPOSHwInfo	POS ハードウェア情報の取得

【プロセス情報】

機能分類	API	機 能
プロセス情報取得	ProcessLog	起動している全プロセスの情報をログファイルに出力する

【操作ログ】

機能分類	API	機 能
POS デバイス操作ログ	OPOS	Scanner/MCR の入力操作ログ
GUI 操作ログ	OCX 組み込みのみ	GUI、標準入力(キー、タッチパネル)の操作ログ

第3章 編集

3.1. イメージ編集 (ActiveX Control)

3.1.1. 概要

イメージ編集コントロール(OCX)は、POS プリンタへのイメージ印刷を支援するためのものです。

3.1.2. オブジェクト形態

オブジェクトタイプ	オブジェクトドライブリプション	プログラマティック ID
ActiveX Control	SHARP IMAGE EDIT CONTROL	SHARP.IEDIT

3.1.3. 機能

イメージ編集コントロールは、ビットマップ画像にグラフィック文字を重ねあわせてファイルに保存することができます。文字はシステムに登録されているフォントを指定することができます。編集イメージを確認することもできます。

3.1.4. 共通 I/F サマリー

プロパティ名称	ドライブリプション	型	アクセス
AutoSize	編集するビットマップのサイズが、ロードするビットマップのサイズに合わせて自動的に変更されるかどうかを指定します	Boolean	R/W
BitmapHeight	画像の高さ(ピクセル単位)を指定します	Single	R/W
BitmapWidth	画像の幅(ピクセル単位)を指定します	Single	R/W
CurrentX	PrintText メソッドの描画の開始点の水平座標(ピクセル単位)を指定します	Single	R/W
CurrentY	PrintText メソッドの描画の開始点の垂直座標(ピクセル単位)を指定します	Single	R/W
Font	描画する文字列のフォントを指定します	Font	R/W
Visible	実行時にコントロールを表示するか非表示にするかを指定します	Boolean	R/W
メソッド名称	ドライブリプション		
LoadBitmap	白黒のビットマップファイルをコントロール内の編集領域に読み込みます		
PrintText	ビットマップ画像上の CurrentX,CurrentY 座標に文字列 Text を描画します		
PrintTextAt	ビットマップ画像上の X,Y 座標に文字列 Text を描画します		
SaveBitmap	編集領域のビットマップ画像を回転させ、モノクロビットマップ形式でファイルに保存します		

3.1.5. プロパティ詳細

AutoSize プロパティ R/W

型 Boolean

説明 編集するビットマップのサイズが、ロードするビットマップのサイズに合わせて自動的に変更されるかどうかを指定します。

値	意味
---	----

True	BitmapHeight プロパティ、BitmapWidth プロパティの値が自動的にビットマップ画像のサイズに変更されます
------	--

False	BitmapHeight プロパティ、BitmapWidth プロパティの値からはみ出した部分は切り捨てられます
-------	--

BitmapHeight プロパティ R/W

型 Single

説明 画像の高さ(ピクセル単位)を指定します。AutoSize プロパティが True の時にビットマップ画像をロードした場合、その画像の高さが設定されます。画像ロード後のプロパティ値変更は、画像ロード直後のみ有効です。

BitmapWidth プロパティ R/W

型 Single

説明 画像の幅(ピクセル単位)を指定します。AutoSize プロパティが True の時にビットマップ画像をロードした場合、その画像の幅が設定されます。画像ロード後のプロパティ値変更は、画像ロード直後のみ有効です。

CurrentX プロパティ R/W

型 Single

説明 PrintText メソッドの描画の開始点の水平座標(ピクセル単位)を指定します。

CurrentY プロパティ R/W

型 Single

説明 PrintText メソッドの描画の開始点の垂直座標(ピクセル単位)を指定します。

Font プロパティ R/W

型 Font

説明 描画する文字列のフォントを指定します。
以下の例は MS 明朝、14 ポイント、ボールドフォントを指定します。
Font.Name = "MS 明朝"
Font.Size = 14
Font.Bold = True

Visible プロパティ R/W

型 Boolean

説明 実行時にコントロールを表示するか非表示にするかを指定します。

3.1.6. メソッド詳細

LoadBitmap メソッド

形式 [VC#] Int32 LoadBitmap(ref String FileName)

[VB.NET] Function LoadBitmap(ByRef FileName As String) As Int32

パラメーター	説明
FileName	ビットマップファイル名を表す文字列を指定します

説明 白黒のビットマップファイルをコントロール内の編集領域に読み込みます。
BitmapHeight、BitmapWidth プロパティの値は AutoSize プロパティの値に依存します。

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	無効なパラメーター、またはデータが異常
PFO_NoExist_Error	ファイルが見つかりません あるいは、ビットマップファイルではありません

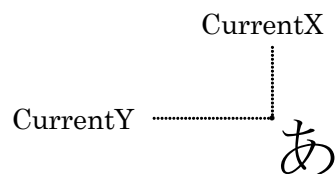
PrintText メソッド

形式 [VC#] Int32 PrintText(ref String Text)

[VB.NET] Function PrintText(ByRef Text As String) As Int32

パラメーター	説明
Text	描画する文字列を指定します

説明 ビットマップ画像上の CurrentX,CurrentY 座標に文字列 Text を描画します。
文字列が“あ”の場合、描画位置関係は下図のようになります。描画されたイメージはビットごとの or になります。
このメソッド実行後 CurrentX プロパティの値は 0 に、CurrentY プロパティの値は文字の高さ※を加えた値に更新されます。



※ 実際の値は文字のポイント数をピクセル数に変換した値です。
戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	無効なパラメーター、またはデータが異常

PrintTextAt メソッド

形式 [VC#] `Int32 PrintTextAt(ref Single X, ref Single Y, ref String Text)`
 [VB.NET] `Function PrintTextAt(ByRef X As Single, ByRef Y As Single, ByRef Text As String) As Int32`

パラメーター	説明
--------	----

<i>X</i>	文字列の描画の開始点の水平座標を示す値(ピクセル単位)です
<i>Y</i>	文字列の描画の開始点の垂直座標を示す値(ピクセル単位)です
<i>Text</i>	描画する文字列を指定します

説明 ビットマップ画像上の *X,Y* 座標に文字列 *Text* を描画します。描画位置関係は `PrintText` メソッドの説明を参照ください。描画されたイメージはビットごとの or になります。
このメソッド実行後 `CurrentX` プロパティの値は 0 に、`CurrentY` プロパティの値は文字の高さ※を加えた値に更新されます。

※ 実際の値は文字のポイント数をピクセル数に変換した値です。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

<code>PFO_Success</code>	正常終了
<code>PFO_Illegal_Error</code>	無効なパラメーター、またはデータが異常

SaveBitmap メソッド

形式 [VC#] `Int32 SaveBitmap(ref String FileName, ref Int16 Rotate)`
 [VB.NET] `Function SaveBitmap(ByRef FileName As String, ByRef Rotate As Int16) As Int32`

パラメーター	説明
--------	----

<i>FileName</i>	保存するビットマップファイル名を表す文字列を指定します
<i>Rotate</i>	画像を反時計回転させた状態で保存する時に指定します 0:0° (回転させない)、1:90°、2:180°、3:270°

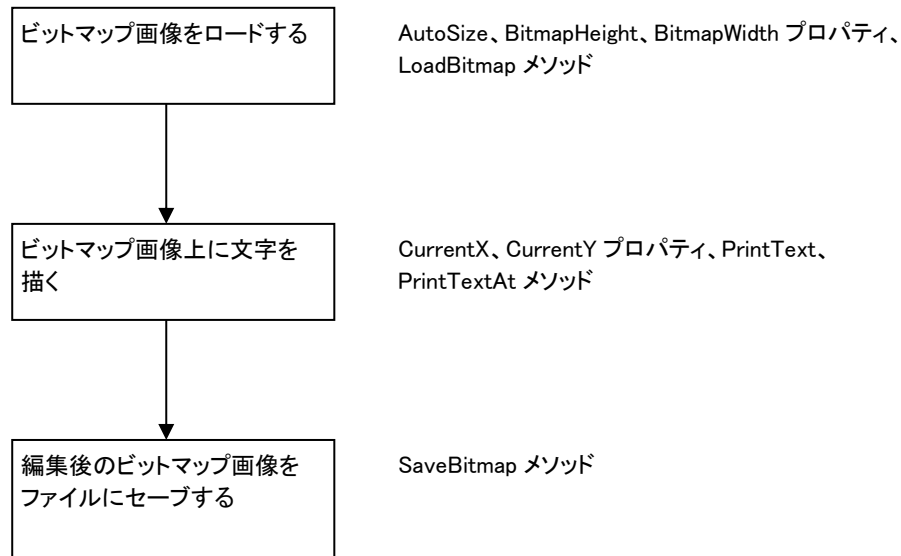
説明 編集領域のビットマップ画像を *Rotate* で指定しただけ反時計回転させ、モノクロビットマップ形式でファイルに保存します。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

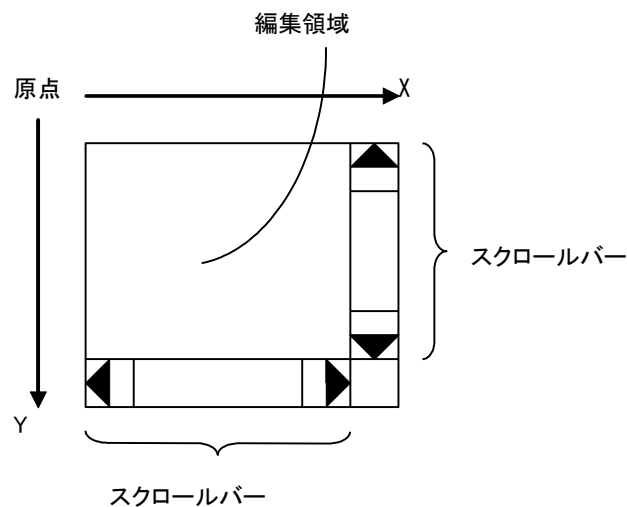
<code>PFO_Success</code>	正常終了
<code>PFO_Illegal_Error</code>	無効なパラメーター、またはデータが異常
<code>PFO_FileWrite_Error</code>	メモリー不足などの予期せぬエラーが発生しました

3.1.7. イメージ編集シーケンス



3.1.8. コントロールの表示と座標系

VC#、VB のフォームにイメージ編集コントロールを貼ると、下図のように表示されます。編集する画像サイズが編集領域のサイズより大きい場合は、スクロールバーでスクロールすることができます。



座標は、イメージの矩形領域の左上を原点とし、左から右に向かって X 座標が増加し、上から下に向かって Y 座標が増加します。

3.1.9. 補足

このコントロールで扱えるイメージデータのサイズは最大(横 2000×縦 2000 ピクセル)です。

第4章 演算/変換

4.1. 概要

演算や各種データ変換を行います。

4.2. モジュール形態

オブジェクトタイプ	オブジェクトドライブリプション	プログラマティック ID
ActiveX EXE	SHARP CALC CONTROL for .NET	SHARP.CALCNET

4.3. 演算・変換 (.NET - ActiveX EXE)

4.3.1. 共通 I/F サマリー

メソッド名称	ドライプリプション
Calcudgt	チェックデジットの計算をします
PStoInt32	符号有り内部 10 進数(PS 型データ)を Int32 型に変換します
NStoInt32	符号有り外部 10 進数(NS 型データ)を Int32 型に変換します
NTtoInt32	符号無し外部 10 進数(N 型データ)を Int32 型に変換します
PStoDecimal	符号有り内部 10 進数(PS 型データ)を Decimal 型に変換します
NStoDecimal	符号有り外部 10 進数(NS 型データ)を Decimal 型に変換します
NtoDecimal	符号無し外部 10 進数(N 型データ)を Decimal 型に変換します
SJIStoJIS	SJIS データを JIS データに変換します
JIStoSJIS	JIS データを SJIS データに変換します
JIS7to8	JIS7 データを JIS8 データに変換します
JIS8to7	JIS8 データを JIS7 データに変換します
JIS8toEBC	JIS8 データを EBCDIC データに変換します
EBCtoJIS8	EBCDIC データを JIS8 データに変換します

4.3.2. メソッド詳細

Calcudgt メソッド

形式 **[VC#]** `Int32 Calcudgt(out Int32 lpDigit, Int32 modType, string data, string weight)`
[VB.NET] `Function Calcudgt(ByRef lpDigit As Int32, ByVal modType As Int32, ByVal data As String, ByVal weight As String) As Int32`

パラメーター	説明
<i>lpDigit</i>	求めたチェックデジットを格納します
<i>modType</i>	モジュールス計算方法 101 : 10-1 モジュールス計算方法を使用 102 : 10-2 モジュールス計算方法を使用 111 : 11-1 モジュールス計算方法を使用 112 : 11-2 モジュールス計算方法を使用 113 : 11-3 モジュールス計算方法を使用 114 : 11-4 モジュールス計算方法を使用 115 : 11-5 モジュールス計算方法を使用
<i>data</i>	対象データ(文字列)
<i>weight</i>	重み数値データ(文字列)

説明 チェックデジットの計算をします。
data に格納されている文字列データ(1~15 桁)に対し、*modType* で指定されているモジュールス計算方法で、チェックデジットを計算します。
weight は重みを表します。各文字は文字コード 0~127 が有効です。

戻り値 モジュールス計算方法については、「付録 B. Calcudgt 計算規則(方法)」を参照してください。
次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	無効なパラメーター、またはデータが異常

例 データが 1234、重みが 1,3,1,3 の場合、VC#から以下のように記述します。

```
LCALCNetLib.CALCClass POSCAL = new LCALCNetLib.CALCClass();
Int32 digit;

String data = "1234";
Char[] weight = { (Char)1, (Char)3, (Char)1, (Char)3 };
int ret = POSCAL.Calcudgt(out digit, 101, data, new String(weight));
```

PStoInt32 メソッド

形式	[VC#]	<code>Int32 PStoInt32(out Int32 lpInt32, String PSData, Int32 size)</code>
	[VB.NET]	<code>Function PStoInt32(ByRef lpInt32 As Int32, ByVal PSData As String, ByVal size As Int32) As Int32</code>
パラメーター	<i>lpInt32</i>	説明
	<i>PSData</i>	計算結果を格納します
	<i>size</i>	PS 型データ(バイトデータ)
		PS 型データのバイト数
説明	<i>PSData</i> に格納されている符号有り内部 10 進数(PS 型データ)を <i>size</i> で指定したバイト数で Int32 型に変換し、 <i>lpInt32</i> が指す領域に格納します。	
	PS 型データについては「付録 C. 演算データ形式」を参照してください。	
	変換に失敗した場合 PFO_Illegal_Error を返しますが、その場合でもできる限り変換後のデータを格納します。以下に、変換に失敗する主な原因と、データの扱いについて記述します。	
	符号が正しく設定されていない → 正の数として扱います 内部 10 進のデータとして 0 から 9 以外のものがある → 0 として扱います データが 10 桁以上ある → 10 桁以上の桁を切り捨てます	
戻り値	次の値のいずれかが戻されます。	
	値	意味
	PFO_Success	正常終了
	PFO_Illegal_Error	無効なパラメーター、またはデータが異常

NStoInt32 メソッド

形式	[VC#]	<code>Int32 NStoInt32(out Int32 lpInt32, String NSData, Int32 size)</code>
	[VB.NET]	<code>Function NStoInt32(ByRef lpInt32 As Int32, ByVal NSData As String, ByVal size As Int32) As Int32</code>
パラメーター	<i>lpInt32</i>	説明
	<i>NSData</i>	計算結果を格納します
	<i>Size</i>	NS 型データ(バイトデータ)
		NS 型データのバイト数
説明	<i>NSData</i> に格納されている符号有り外部 10 進数(NS 型データ)を <i>size</i> で指定したバイト数で Int32 型に変換し、 <i>lpInt32</i> が指す領域に格納します。	
	NS 型データについては「付録 C. 演算データ形式」を参照してください。	
	変換に失敗した場合 PFO_Illegal_Error を返しますが、その場合でもできる限り変換後のデータを格納します。以下に、変換に失敗する主な原因と、データの扱いについて記述します。	
	符号が正しく設定されていない → 正の数として扱います 外部 10 進のデータとして 0 から 9 以外のものがある → 0 として扱います データが 10 桁以上ある → 10 桁以上の桁を切り捨てます	
戻り値	次の値のいずれかが戻されます。	
	値	意味
	PFO_Success	正常終了
	PFO_Illegal_Error	無効なパラメーター、またはデータが異常

NtoInt32 メソッド

形式	[VC#]	<code>Int32 NtoInt32(out Int32 lpInt32, String NData, Int32 size)</code>
	[VB.NET]	<code>Function NtoInt32(ByRef lpInt32 As Int32, ByVal NData As String, ByVal size As Int32) As Int32</code>
	パラメーター	説明
	<i>lpInt32</i>	計算結果を格納します
説明	<i>NData</i>	N 型データ(バイトデータ)
	<i>Size</i>	N 型データのバイト数
	<i>NData</i> に格納されている符号無し外部 10 進数(N 型データ)を <i>size</i> で指定したバイト数で Int32 型に変換し、 <i>lpInt32</i> が指す領域に格納します。 N 型データについては「付録 C. 演算データ形式」を参照してください。 変換に失敗した場合 PFO_Illegal_Error を戻しますが、その場合でもできる限り変換後のデータを格納します。以下に、変換に失敗する主な原因と、データの扱いについて記述します。 外部 10 進のデータとして 0 から 9 以外のものがある → 0 として扱います データが 10 桁以上ある → 10 桁以上の桁を切り捨てます	
戻り値	次の値のいずれかが戻されます。	
	値	意味
	PFO_Success	正常終了
	PFO_Illegal_Error	無効なパラメーター、またはデータが異常

PStoDecimal メソッド

形式	[VC#]	<code>Int32 PStoDecimal(out Decimal lpDecimal, String PSData, Int32 size)</code>
	[VB.NET]	<code>Funciton PStoDecimal(ByRef lpDecimal As Decimal, ByVal PSData As String, ByVal size As Int32) As Int32</code>
	パラメーター	説明
	<i>lpDeciaml</i>	計算結果を格納します
説明	<i>PSData</i>	PS 型データ(バイトデータ)
	<i>Size</i>	PS 型データのバイト数
	<i>PSData</i> に格納されている符号有り内部 10 進数(PS 型データ)を <i>size</i> で指定したバイト数で Decimal 型に変換し、 <i>lpDecimal</i> が指す領域に格納します。 PS 型データについては「付録 C. 演算データ形式」を参照してください。 変換に失敗した場合 PFO_Illegal_Error を戻しますが、その場合でもできる限り変換後のデータを格納します。以下に、変換に失敗する主な原因と、データの扱いについて記述します。 符号が正しく設定されていない → 正の数として扱います 内部 10 進のデータとして 0 から 9 以外のものがある → 0 として扱います データが Decimal 型で表現できる数値よりも大きい → 表現できる桁数分だけ下位の方からデータを設定します	
戻り値	次の値のいずれかが戻されます。	
	値	意味
	PFO_Success	正常終了
	PFO_Illegal_Error	無効なパラメーター、またはデータが異常

NSDecimal メソッド

形式	[VC#]	<code>Int32 NSDecimal(out Decimal lpDecimal, String NSData, Int32 size)</code>
	[VB.NET]	<code>Function NSDecimal(ByRef lpDecimal As Decimal, ByVal NSData As String, ByVal size As Int32) As Int32</code>
	パラメーター	説明
	<i>lpDeciaml</i>	計算結果を格納します
	<i>NSData</i>	NS 型データ(バイトデータ)
	<i>size</i>	NS 型データのバイト数
説明	<i>NSData</i> に格納されている符号有り外部 10 進数(NS 型データ)を <i>size</i> で指定したバイト数で <i>Decimal</i> 型に変換し、 <i>lpDecimal</i> が指す領域に格納します。	
	NS 型データについては「付録 C. 演算データ形式」を参照してください。	
	変換に失敗した場合 <i>PFO_Illegal_Error</i> を戻しますが、その場合でもできる限り変換後のデータを格納します。以下に、変換に失敗する主な原因と、データの扱いについて記述します。	
	符号が正しく設定されていない	→ 正の数として扱います
	内部 10 進のデータとして 0 から 9 以外のものがある	→ 0 として扱います
	データが <i>Decimal</i> 型で表現できる数値よりも大きい	→ 表現できる桁数分だけ下位の方からデータを設定します
戻り値	次の値のいずれかが戻されます。	
	値	意味
	<i>PFO_Success</i>	正常終了
	<i>PFO_Illegal_Error</i>	無効なパラメーター、またはデータが異常

NtoDecimal メソッド

形式	[VC#]	<code>Int32 NtoDecimal(out Decimal lpDecimal, String NData, Int32 size)</code>
	[VB.NET]	<code>Function NtoDecimal(ByRef lpDecimal As Decimal, ByVal NData As String, ByVal size As Int32) As Int32</code>
	パラメーター	説明
	<i>lpDeciaml</i>	計算結果を格納します
	<i>NData</i>	N 型データ(バイトデータ)
	<i>size</i>	N 型データのバイト数
説明	<i>NData</i> に格納されている符号無し外部 10 進数(N 型データ)を <i>size</i> で指定したバイト数で <i>Decimal</i> 型に変換し、 <i>lpDecimal</i> が指す領域に格納します。	
	N 型データについては「付録 C. 演算データ形式」を参照してください。	
	変換に失敗した場合 <i>PFO_Illegal_Error</i> を戻しますが、その場合でもできる限り変換後のデータを格納します。以下に、変換に失敗する主な原因と、データの扱いについて記述します。	
	内部 10 進のデータとして 0 から 9 以外のものがある	→ 0 として扱います
	データが <i>Decimal</i> 型で表現できる数値よりも大きい	→ 表現できる桁数分だけ下位の方からデータを設定します
戻り値	次の値のいずれかが戻されます。	
	値	意味
	<i>PFO_Success</i>	正常終了
	<i>PFO_Illegal_Error</i>	無効なパラメーター、またはデータが異常

SJIStoJIS メソッド

形式	[VC#]	Int32 SJIStoJIS(out String <i>jis</i> , String <i>sjis</i>)
	[VB.NET]	Function SJIStoJIS(ByRef <i>jis</i> As String, ByVal <i>sjis</i> As String) As Int32
説明	パラメーター	説明
	<i>Jis</i>	変換データを格納します
説明	<i>Sjis</i>	変換元 SJIS コード列
	<i>sjis</i> に格納されている SJIS データを JIS データに変換し、 <i>jis</i> が指す領域に格納します。変換できないデータを検出した場合は変換を中断し、変換済みデータのみを <i>jis</i> が指す領域に格納し、エラーコードを戻します。	
戻り値	次の値のいずれかが戻されます。	
	値	意味
	PFO_Success	正常終了
	PFO_Illegal_Error	無効なパラメーター、またはデータが異常

JIStoSJIS メソッド

形式	[VC#]	Int32 JIStoSJIS(out String <i>sjis</i> , String <i>jis</i>)
	[VB.NET]	Function JIStoSJIS(ByRef <i>sjis</i> As String, ByVal <i>jis</i> As String) As Int32
説明	パラメーター	説明
	<i>sjis</i>	変換データを格納します
説明	<i>Jis</i>	変換元 JIS コード列
	<i>jis</i> に格納されている JIS データを SJIS データに変換し、 <i>sjis</i> が指す領域に格納します。変換できないデータを検出した場合は変換を中断し、変換済みデータのみを <i>sjis</i> が指す領域に格納し、エラーコードを戻します。 JIS コードのエスケープシーケンスは変換できないデータとして扱います。	
戻り値	次の値のいずれかが戻されます。	
	値	意味
	PFO_Success	正常終了
	PFO_Illegal_Error	無効なパラメーター、またはデータが異常

JIS7toJIS8 メソッド

形式	[VC#]	Int32 JIS7to8(out String <i>jis8</i> , out Int32 <i>jis8size</i> , String <i>jis7</i> , Int32 <i>jis7size</i>)
	[VB.NET]	Function JIS7to8(ByRef <i>jis8</i> As String, ByRef <i>jis8size</i> As Int32, ByVal <i>jis7</i> As String, ByVal <i>jis7size</i> As Int32) As Int32
説明	パラメーター	説明
	<i>jis8</i>	変換データを格納します
説明	<i>jis8size</i>	変換データの有効バイト数を格納します
	<i>jis7</i>	変換元 JIS7 コード列
説明	<i>jis7size</i>	変換元 JIS7 コード列のバイト数
	<i>jis7</i> に格納されている JIS7 データを JIS8 データに変換し、変換データを <i>jis8</i> が指す領域に、その有効バイト数を <i>jis8size</i> が指す領域に格納します。	
戻り値	次の値のいずれかが戻されます。	
	値	意味
	PFO_Success	正常終了
	PFO_Illegal_Error	無効なパラメーター、またはデータが異常

JIS8toJIS7 メソッド

形式 [VC#] `Int32 JIS8to7(out String jis7, out Int32 jis7size, String jis8, Int32 jis8size)`
 [VB.NET] `Function JIS8to7(ByRef jis7 As String, ByRef jis7size As Int32, ByVal jis8 As String, ByVal jis8size As Int32) As Int32`

パラメーター	説明
<i>jis7</i>	変換データを格納します
<i>jis7size</i>	変換データの有効バイト数を格納します
<i>jis8</i>	変換元 JIS8 コード列
<i>jis8size</i>	変換元 JIS8 コード列のバイト数
説明	<i>jis8</i> に格納されている JIS8 データを JIS7 データに変換し、変換データを <i>jis7</i> が指す領域に、その有効バイト数を <i>jis7size</i> が指す領域に格納します。
戻り値	次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	無効なパラメーター、またはデータが異常

JIS8toEBC メソッド

形式 [VC#] `Int32 JIS8toEBC(out String ebcdic, out Int32 ebcdicsize, String jis8, Int32 jis8size)`
 [VB.NET] `Function JIS8toEBC(ByRef ebcdic As String, ByRef ebcdicsize As Int32, ByVal jis8 As String, ByVal jis8size As Int32) As Int32`

パラメーター	説明
<i>ebcdic</i>	変換データを格納します
<i>ebcdicsize</i>	変換データの有効バイト数を格納します
<i>jis8</i>	変換元 JIS8 コード列
<i>jis8size</i>	変換元 JIS8 コード列のバイト数
説明	<i>jis8</i> に格納されている JIS8 データを EBCDIC データに変換し、変換データを <i>ebcdic</i> が指す領域に、その有効バイト数を <i>ebcdicsize</i> が指す領域に格納します。 変換方式は「付録 D. 演算変換テーブル」を参照ください。
戻り値	次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	無効なパラメーター、またはデータが異常

形式	[VC#]	Int32 EBCToJIS8(out String jis8, out Int32 jis8size, String ebcdic, Int32 ebcdicsize)
	[VB.NET]	Function EBCToJIS8(ByRef jis8 As String, ByRef jis8size As Int32, ByVal ebcdic As String, ByVal ebcdicsize As Int32) As Int32

説明 *ebcdic* に格納されている EBCDIC データを JIS8 データに変換し、変換データを *jis8* が指す領域に、その有効バイト数を *jis8size* が指す領域に格納します。
変換方式は「付録 D. 演算変換テーブル」を参照ください。

SJIStoJIS、JIStoSJIS、JIS7toJIS8、JIS8toJIS7、JIS8toEBC、EBCtoJIS8 関数群は、過去互換を保つために用意されています。.NET 環境下で文字コード変換を行う場合、System.Text.Encoding.Convert 関数をご利用ください。

文字コード	コードページ
SJIS	932
JIS	50220
JIS7	50222
JIS8	932
EBCDIC	20290

```
System.Text.Encoding eSJIS = System.Text.Encoding.GetEncoding(932);
System.Text.Encoding eJIS = System.Text.Encoding.GetEncoding(50220);
byte[] bSrc = { SJIS 文字データ };
byte[] bDst = System.Text.Encoding.Convert(eSJIS, eJIS, bSrc);
```

26

サンプルプログラム(SJIStoJIS 関数使用例) :

```
//入力文字列
byte[] bSJIS = { SJIS 文字データ };
//出力文字(string 型)
string strBuffer;

//バイトスワップを実施
int nSize = bSJIS.Length / 2 + bSJIS.Length % 2;
char[] cSJIS = new char[nSize];
for (int i = 0; i < bSJIS.Length/2 + bSJIS.Length %2; i++)
{
    cSJIS[i] = (char)bSJIS[i * 2];
    cSJIS[i] |= (char)(bSJIS[i*2 + 1] << 8);
}

//string 型(Unicode/リトルエンディアン)に変換し、PFO で文字コード変換
//calc: 本コントロールのインスタンス
int nRet = calc.SJIStoJIS(out strBuffer, new string(cSJIS));
//データを取り出す
byte[] bJIS = System.Text.Encoding.Unicode.GetBytes(strBuffer);
```

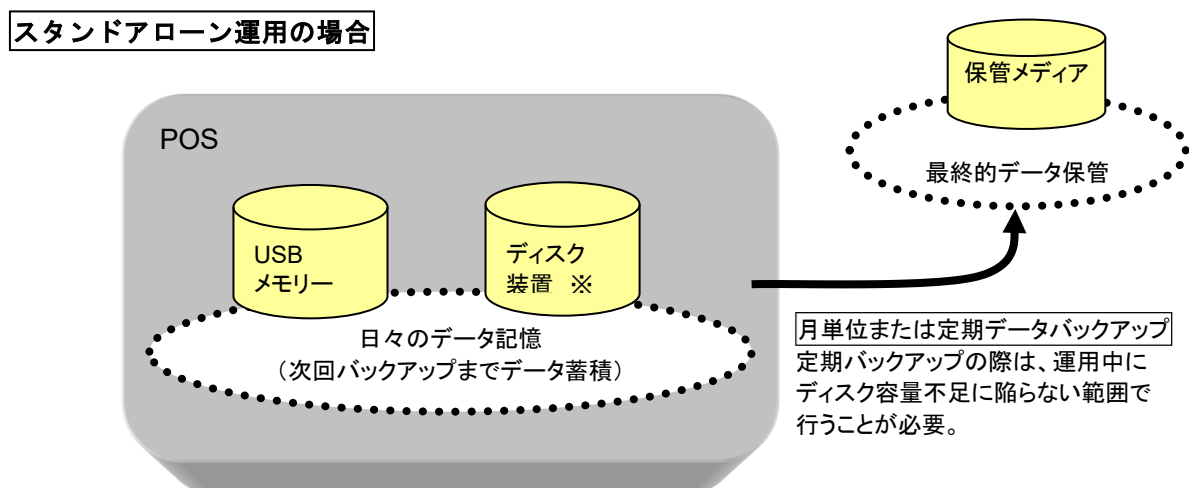
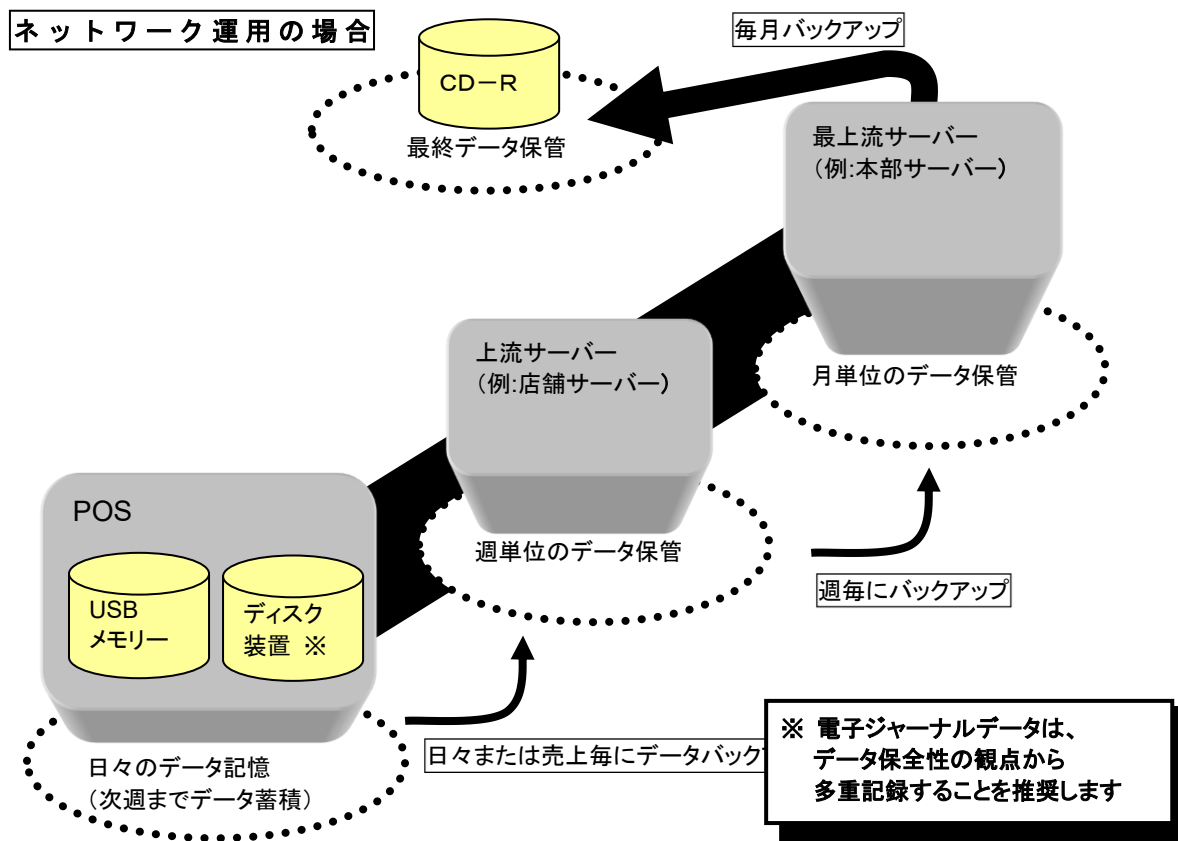
第5章 電子ジャーナル

5.1. 電子ジャーナル(ActiveX Control)

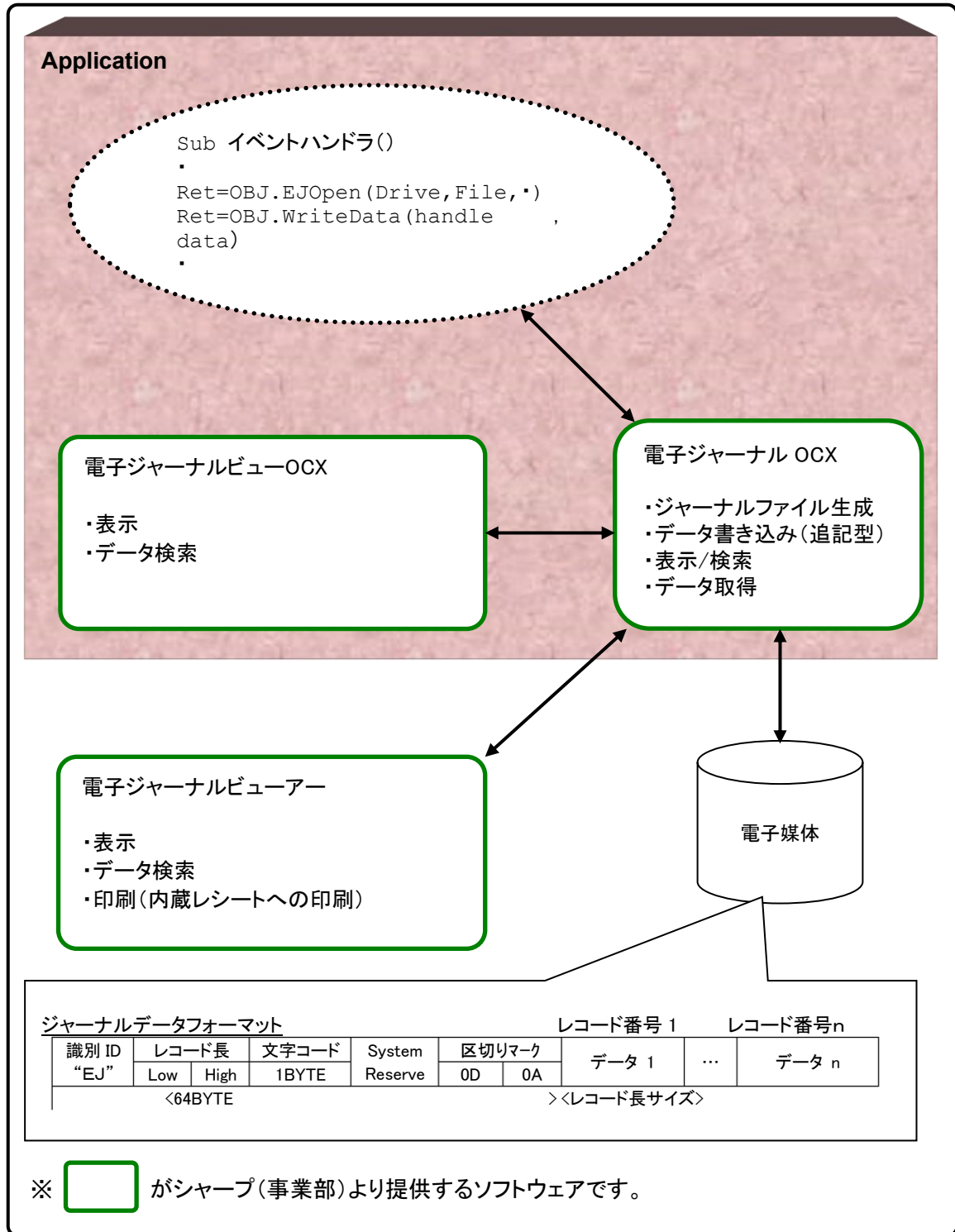
5.1.1. 概要

ジャーナルデータの電子化を行います。機能として、ジャーナルデータの電子媒体(ドライブ装置)への記憶や、記憶データの表示・検索・印刷機能、ジャーナルファイル診断・修復をサポートしています。

【ジャーナル運用想定図】



【電子ジャーナル ソフトウェア概念図】

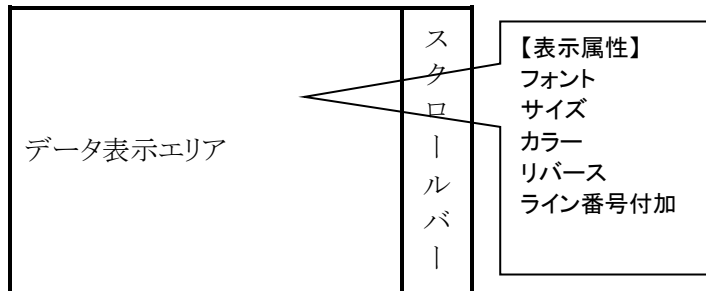


【機能】

- ・ジャーナルファイル生成 (API)
- ・ジャーナルデータ書き込み (API)
- ・ジャーナルデータ表示とスクロール (API/UI)
- ・ジャーナルデータ検索 (API)
- ・ジャーナルデータ読み出し (API)
- ・ジャーナルファイルの削除 (API)
- ・ジャーナルファイル検索 (API)

【UI 仕様】

表示仕様

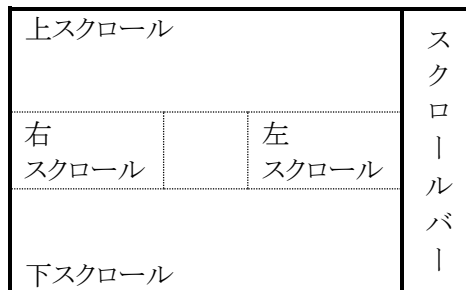


入力仕様

データスクロールは、下記のビュー領域内のクリック/ドラッグ操作およびスクロールバーのいずれでも可能です。

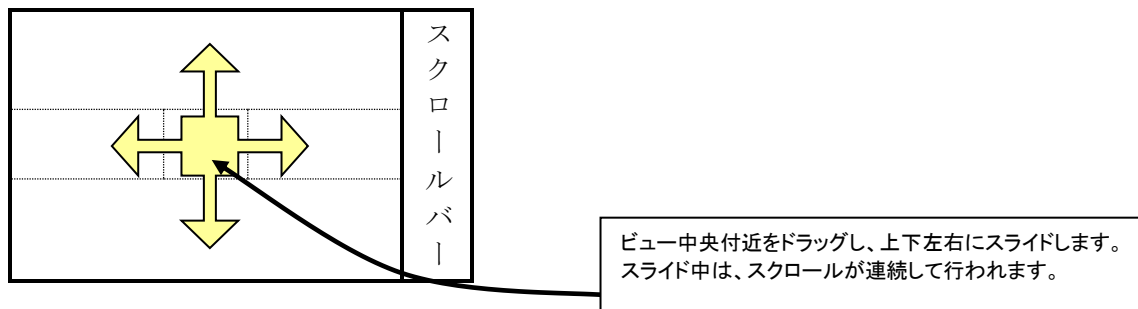
【クリック操作によるスクロール指定】

下記ポジションに対するクリック(マウスのクリック操作)で、データスクロールが行われます。スクロールは、横:ビュー横サイズの 1/2、縦:1 ライン単位となります。



【ドラッグ操作によるスクロール指定】

ビュー中央付近をドラッグし、上下左右に移動すると連続的にスクロールが行われます。本操作は、クリック操作とは異なり移動量分のスクロールが可能です。



5.1.2. オブジェクト形態

オブジェクトタイプ	オブジェクトドライブリプション	プログラマティック ID
ActiveX Control	SHARP ELECTRONIC JOURNALCONTROL	SHARP.EJOURNAL

5.1.3. 共通 I/F サマリー

プロパティ名称	ドライブリプション	型	アクセス
BackColor	背景色の指定	OLE_COLOR	R/W
BackType	背景モードの指定	Long	R/W
BackImage	背景イメージの指定	PICTURE	R/W
Font	フォント指定	FONT	R/W
ForeColor	フォアグラウンドカラー指定	OLE_COLOR	R/W
LineNumDisp	ライン番号の表示指定	Boolean	R/W
ReverseLine	リバース表示指定	Long	R/W
Visible	表示領域可視/不可視	Boolean	R/W
UIMode	UI モード指定	Long	R/W
ViewPosition	表示位置指定	Long	R/W
ViewCounts	表示有効ライン数	Long	R
メソッド名称	ドライブリプション		
EJCreate	ジャーナルファイル生成		
EJOpen	ジャーナルファイルのオープン		
EJClose	ジャーナルファイルのクローズ		
EJDelete	ジャーナルファイルの削除		
EJFileFind	ジャーナルファイルの検索		
WriteData	ジャーナルデータ(テキスト)の書き込み(追記記憶)		
SearchData	任意位置からの検索		
ReadData	ジャーナルデータ取得		
ViewData	ジャーナルデータの表示		
ViewClear	ジャーナル表示データのクリア		
RecordSize	ジャーナルデータのレコードサイズを取得		
RecordCounts	ジャーナルデータのレコード数を取得		

5.1.4. プロパティ詳細

BackColor プロパティ R/W

型 OLE_COLOR

説明 ビューの背景色を指定します。
デフォルトは、ボタンの表面色に設定されています。

BackType プロパティ R/W

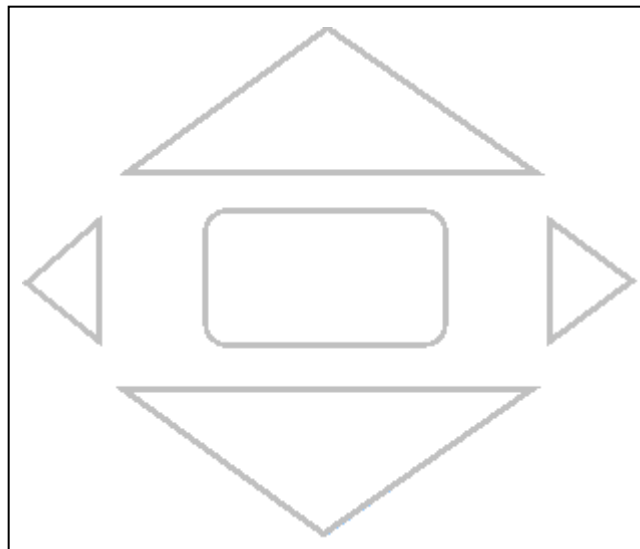
型 Long

説明 背景タイプを指定します。デフォルトは PFO_EJ_BackImage です。
PFO_EJ_BackColor : 背景を BackColor で塗りつぶします。
PFO_EJ_BackImage : BackImage プロパティに設定されたイメージを表示します。

BackImage プロパティ R/W

型 PICTURE

説明 背景イメージ(BMP)を指定します。
設定可能なイメージファイルは BMP ファイルで、ICO ファイルはサポートされません。
デフォルトでは、以下のようなイメージが設定されています。



Font プロパティ R/W

型 FONT

説明 ジャーナルデータの表示フォントを指定します。

ForeColor プロパティ R/W

型 OLE_COLOR

説明 テキストの表示色を指定します。

LineNumDisp プロパティ R/W

型 Boolean

説明 ライン番号の表示モードを設定します。
TRUE : 表示
FALSE : 非表示

ReverseLine プロパティ R/W

型 Long

説明 リバース表示する行番号を指定します。

Visible プロパティ R/W

型 Boolean

説明 表示領域の可視、不可視を設定します。

TRUE : 可視

FALSE : 不可視

UIMode プロパティ R/W

型 Long

説明 UI(ユーザーインタフェース)のスクロールバー表示モードを設定します。

デフォルトは、PFO_EJ_UIMode_EnableS です。

PFO_EJ_UIMode_Disable : UIからの入力禁止(スクロールバー非表示)

PFO_EJ_UIMode_Enable : UIからの入力許可(スクロールバー非表示)

PFO_EJ_UIMode_EnableS : UIからの入力許可(スクロールバー表示)

ViewPosition プロパティ R/W

型 Long

説明 ジャーナルデータの表示レコードを指定・取得します。

本プロパティに設定される値が、表示領域の最上位に表示されます。

ViewData メソッド実行後、本プロパティは、1 に設定されます。

ViewCounts プロパティ R

型 Long

説明 表示領域に表示可能なライン数を示します。

5.1.5. メソッド詳細

EJCreate メソッド

形式 [VC#] `Int32 EJCreate(String Drive, String JournalName, Int32 RecordSize, Int32 CreateMode)`
 [VB.NET] `Function EJCreate(ByVal Drive As String, ByVal JournalName As String, ByVal RecordSize As Int32, ByVal CreateMode As Int32) As Int32`

パラメーター	説明
<i>Drive</i>	生成するジャーナルファイルのドライブ名を指定します。 例 "C:"
<i>JournalName</i>	ジャーナルファイル名を指定します。 例 "EJ990707.001"
<i>RecordSize</i>	ジャーナルファイルは、 <i>Drive</i> ¥ <i>Ejournal</i> フォルダに生成されます。 1レコード当たりの長さ(バイト長)を指定します。
<i>CreateMode</i>	生成するファイル属性を指定します。 PFO_EJ_CreateNormal : 標準 PFO_EJ_CreateHidden : 隠し属性

説明 ジャーナルファイルが生成されます。
 ジャーナルファイルは、*Drive*+"¥*Ejournal*¥"+*JournalName* の規則で作成されます。
 また、生成されたジャーナルファイルは、リードオンリー属性です。
 (オープン中は一時的にリードライトモードになります。)

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正
PFO_Exist_Error	ファイルが既に存在している
PFO_FileSpace_Error	書き込みスペースがない
PFO_FileProtect_Error	プロテクトエラー
PFO_FileWrite_Error	ライトエラー(システムエラー)
PFO_Device_Error	デバイス(またはメディア)が存在しない

EJOpen メソッド

形式 **[VC#]** `Int32 EJOpen(String Drive, String JournalName, Int32 OpenMode, ref Int32 Handle)`
[VB.NET] `Function EJOpen(ByVal Drive As String, ByVal JournalName As String, ByVal OpenMode As Int32, ByRef Handle As Int32) As Int32`

パラメーター	説明
<i>Drive</i>	生成するジャーナルファイルのドライブ名を指定します。 例 "C:"
<i>JournalName</i>	ジャーナルファイル名を指定します。
<i>OpenMode</i>	オープンするモードを指定します。 PFO_EJ_WriteMode : ライトモード (書き込み単位でフラッシュアウト) PFO_EJ_WriteBufferMode : ライトモード(バッファ使用) PFO_EJ_ReadMode : リードモード <u>ライトモードで利用できるメソッド</u> EJxxx, WriteData, RecordSize, RecordCounts <u>リードモードで利用できるメソッド</u> EJxxx, SearchData, GetData, ViewData, RecordSize, RecordCounts
<i>Handle</i>	オープンされたハンドルが戻ります。

説明 ジャーナルファイルをオープンします。
 オープンは、排他的(占有)に行われますので、既オープン of ジャーナルファイルに対して、多重オープンすることは出来ません。
 オープンするジャーナルファイルは事前に EJCreate メソッドで生成しておく必要があります。
 PFO_EJ_WriteMode モードは、WriteData メソッドと同期して物理デバイスへ書き込みが行われます。一方、PFO_EJ_WriteBufferMode モードは、バッファを介して書き込みが行われるモードで、バッファ更新時または EJClose メソッドにて物理デバイスへの書き込みが行われます。
USB メモリー利用時の注意
 USB メモリー(フラッシュメモリ)は、同一セクターへの書き込み回数に制限があります。
 PFO_EJ_WriteMode モードは、WriteData メソッド毎に USB メモリーへの書き込みが生じるため、PFO_EJ_WriteBufferMode に比べ同一セクターへの書き込み回数が増加し、USB メモリーの寿命を縮める事になりますので注意が必要です。

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正
PFO_Exist_Error	既にオープンされています(多重オープン)
PFO_NoExist_Error	ファイルが存在しない
PFO_FileFormat_Error	ファイルフォーマットエラー
PFO_FileProtect_Error	プロテクトエラー
PFO_FileWrite_Error	ライトエラー(システムエラー)
PFO_Device_Error	デバイス(またはメディア)が存在しない

EJClose メソッド

形式 [VC#] Int32 EJClose(Int32 *Handle*)

[VB.NET] Function EJClose(ByVal *Handle* As Int32) As Int32

パラメーター	説明
--------	----

<i>Handle</i>	クローズするハンドルを指定。
---------------	----------------

説明 指定されたジャーナルファイルを削除します。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

PFO_Success	正常終了
PFO_Illegal_Error	ファイルハンドル不正
PFO_FileSpace_Error	書き込みスペースがない
PFO_FileProtect_Error	プロテクトエラー
PFO_FileWrite_Error	ライトエラー(システムエラー)

EJDelete メソッド

形式 [VC#] Int32 EJDelete(String *Drive*, String *JournalName*, Int32 *DeleteMode*)

[VB.NET] Function EJDelete(ByVal *Drive* As String, ByVal *JournalName* As String, ByVal *DeleteMode* As Int32) As Int32

パラメーター	説明
--------	----

<i>Drive</i>	削除するジャーナルファイルのドライブ名を指定します。
--------------	----------------------------

例 “C:”

<i>JournalName</i>	ジャーナル名を指定します。
--------------------	---------------

オープン時に設定したファイル名を指定します。

<i>DeleteMode</i>	ジャーナルファイルの削除モードを指定します。
-------------------	------------------------

PFO_EJ_DeleteOld : *JournalName* は無視され、更新日付の最も古いジャーナルファイル(*.ejd)を削除

PFO_EJ_DeleteSel : *JournalName* に指定されたジャーナルファイルを削除

説明 指定されたジャーナルファイルを削除します。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正
PFO_Exist_Error	オープン中であり、削除できません
PFO_NoExist_Error	ファイルが存在しない
PFO_FileProtect_Error	プロテクトエラー
PFO_FileWrite_Error	ライトエラー(システムエラー)
PFO_Device_Error	デバイス(またはメディア)が存在しない

EJFileFind メソッド

形式 **[VC#]** `Int32 EJFileFind(String Drive, ref String Data, Int32 Mode)`
 [VB.NET] `Function EJFileFind(ByVal Drive As String, ByRef Data As String, ByVal Mode As Int32) As Int32`

パラメーター	説明
<i>Drive</i>	ファイル検索ドライブ
<i>Data</i>	検出したジャーナルファイル名が格納されます “”場合は、ファイル無し
<i>Mode</i>	ファイル検索モード PFO_EJ_FindFirst : 最初のファイル検索 PFO_EJ_FindNext : 次のファイル検索 (<i>Drive</i> は設定不要)

説明 指定されたドライブ内に存在するジャーナルファイルを検索します。
 検索は、ファイル属性(隠し属性)に関係なく実行されます。

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正

WriteData メソッド

形式 **[VC#]** `Int32 WriteData(Int32 Handle, String Data)`
 [VB.NET] `Function WriteData(ByVal Handle As Int32, ByVal Data As String) As Int32`

パラメーター	説明
<i>Handle</i>	オープンハンドル
<i>Data</i>	書き込みデータ

説明 指定されたデータを書き込みます。データは、最終レコードの次に追加されます。
 データはテキスト文字列データです。
 バイナリーデータ(NULL コードを除く)を書き込んだ場合、ビュー表示は保証されません。

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正 (ハンドル, オープンモードが違う)
PFO_FileSpace_Error	書き込みスペースがない
PFO_FileProtect_Error	プロテクトエラー
PFO_FileWrite_Error	ライトエラー (システムエラー)

SerchData メソッド

形式 [VC#] `Int32 SearchData(Int32 Handle, String Data, Int32 Position, Boolean Direction, ref Int32 Record)`
[VB.NET] `Function SearchData(ByVal Handle As Int32, ByVal Data As String, ByVal Position As Int32, ByVal Direction As Boolean, ByRef Record As Int32) As Int32`

パラメーター	説明
<i>Handle</i>	オープンハンドル
<i>Data</i>	検索文字列を指定します。
<i>POSiTion</i>	検索開始レコード番号を指定します (1～RecordCounts)
<i>Direction</i>	検索方向 True : 新データ方向 False : 過去データ方向に検索
<i>Record</i>	検索結果(該当レコード番号)が格納されます 0 の場合、該当データ無しを意味します

説明 ジャーナルファイルから指定された文字列を検索します。

検索結果は、Record 引数に戻されます。

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正(ハンドル, オープンモードが違う)
PFO_FileRead_Error	リードエラー(システムエラー)

ReadData メソッド

形式 [VC#] `Int32 ReadData(Int32 Handle, ref String Data, Int32 Record)`
[VB.NET] `Function ReadData(ByVal Handle As Int32, ByRef Data As String, ByVal Record As Int32) As Int32`

パラメーター	説明
<i>Handle</i>	オープンハンドル
<i>Data</i>	取得するデータ格納ポインタ
<i>Record</i>	データ取得レコード

説明 任意レコードのジャーナルデータを取得します。

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正(ハンドル, オープンモードが違う)
PFO_FileRead_Error	リードエラー(システムエラー)

ViewData メソッド

形式 [VC#] `Int32 ViewData(Int32 Handle)`
[VB.NET] `Function ViewData(ByVal Handle As Int32) As Int32`

パラメーター	説明
--------	----

<i>Handle</i>	オープンハンドル
---------------	----------

説明 指定されたジャーナルデータをメモリ上にロードし、ビューに表示します。
再度本メソッドが呼び出された場合、直前の表示データはメモリから削除され、新たに指定されたデータに更新されます。
本メソッド実行時、ViewPosition プロパティは 1 に設定され、ジャーナルデータの先頭より表示されます。

ジャーナルデータ中に 20h 以下のコードが含まれている場合、それらのコードは 20h(スペース)に置き換えられ表示されます。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

PFO_Success	正常終了
-------------	------

PFO_Illegal_Error	パラメーター不正(ハンドル, オープンモードが違う)
-------------------	----------------------------

PFO_FileRead_Error	リードエラー(システムエラー)
--------------------	-----------------

ViewClear メソッド

形式 [VC#] `Int32 ViewClear()`
[VB.NET] `Function ViewClear() As Int32`

説明 ViewData メソッドで表示している内容をクリア(表示用バッファから削除)します。
ビューエリアの表示を消す場合に利用します。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

PFO_Success	正常終了
-------------	------

RecordSize メソッド

形式 [VC#] `Int32 RecordSize(Int32 Handle, ref Int32 Size)`
[VB.NET] `Function RecordSize(ByVal Handle As Int32, ByRef Size As Int32) As Int32`

パラメーター	説明
--------	----

<i>Handle</i>	オープンハンドル
---------------	----------

<i>Size</i>	レコードサイズが格納されます
-------------	----------------

説明 指定されたジャーナルデータのレコードサイズを取得します。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

PFO_Success	正常終了
-------------	------

PFO_Illegal_Error	パラメーター不正(ハンドル, オープンモードが違う)
-------------------	----------------------------

PFO_FileRead_Error	リードエラー(システムエラー)
--------------------	-----------------

RecordCounts ソッド

形式 **[VC#]** `Int32 RecordCounts (Int32 Handle, ref Int32 Count)`
[VB.NET] `Function RecordCounts (ByVal Handle As Int32, ByRef Count As Int32) As Int32`

パラメーター	説明
<i>Handle</i>	オープンハンドル
<i>Count</i>	レコード数が格納されます

説明 指定されたジャーナルデータのレコード数を取得します。
戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_Illegal_Error	パラメーター不正 (ハンドル, オープンモードが違う)
PFO_FileRead_Error	リードエラー (システムエラー)

5.1.6. 設定ファイル (PFOEJCODE.ini)

【設定ファイル (PFOEJCODE.ini) の格納場所】

ファイル名 PFOEJCODE.ini
格納先フォルダ名 ¥Windows¥SysWOW64

内容	未指定時
電子ジャーナルファイルの文字コードの指定	SJIS (Shift JIS)
UTF16: UNICODE	
その他: Shift JIS	

5.1.7. 特記事項

- ・ジャーナルファイル属性について
ジャーナルファイルは、生成 (EJCreate メソッド) 時に隠し属性に設定することができます。
ジャーナルファイルを隠し属性で生成した場合、エクスプローラ等でファイル所在を確認できませんので、誤ってフォーマットしないなどの注意が必要です。
- ・ジャーナルファイルの格納フォルダについて
EJCreate メソッドは、¥EJOURNAL フォルダ内にジャーナルファイルを生成します。指定されたドライブに¥EJOURNAL フォルダが存在しない場合、EJCreate メソッド内で自動的にフォルダ生成 (非隠し属性) されます。フォルダを隠し属性にしたい場合は、前もってエクスプローラなどで隠し属性の¥EJOURNAL フォルダを作成しておいてください。
- ・ドライブ名について
ローカルドライブ名およびネットワークドライブ名を指定することができます。
- ・PFO_FileFormat_Error (ファイルフォーマットエラー) について
PFO_FileFormat_Error は、ジャーナルファイルの先頭に識別 ID “EJ” が格納されていないファイルにアクセスした場合に発生します。
- ・電子ジャーナル文字コード指定について
設定ファイル POSEJCODE.ini を UTF16 に指定することで電子ジャーナルは JIS X 0213:2004 (第三/第四水準文字) に対応します。
- JIS X 0213:2004 に対応した電子ジャーナルファイルと、対応前の電子ジャーナルファイル (POSEJCODE.ini を MBCS に設定して作成した電子ジャーナルファイル) の混在はできません。
- POS システム以外の PC (OS : Windows XP professional) で JIS X 0213:2004 の電子ジャーナルファイルを閲覧する場合、Microsoft より提供されている JIS X 0213:2004 用のフォントファイルをインストールする必要があります。
- JIS X 0213:2004 (第三/第四水準文字) 以外の文字は対応しません。
- UTF16 では半角文字も 2 バイト使用します。

5.2. .NET 版 電子ジャーナルビューアー(.NET EXE)

5.2.1. 概要

電子ジャーナルビューアーは、電子ジャーナルモジュールで書き込まれたジャーナルデータの表示、検索、印刷機能を持ったツール(.NET アプリケーション)です。単独実行または POS アプリケーションから呼び出して利用することができます。

なお、本ツールのカスタマイズを可能とするために、プロジェクトファイル(C#ソース、VB.Net ソース)も提供しています。

(注意)

- ・本アプリケーションは、PC での動作も可能です。OS は、Windows 10 または Windows 11、.NET Framework 3.5 または 4.0/4.5/4.6/4.7/4.8.1 がインストールされている必要があります。
- ・PC で動作させた場合、印刷機能はサポートされません。

【基本機能概要】

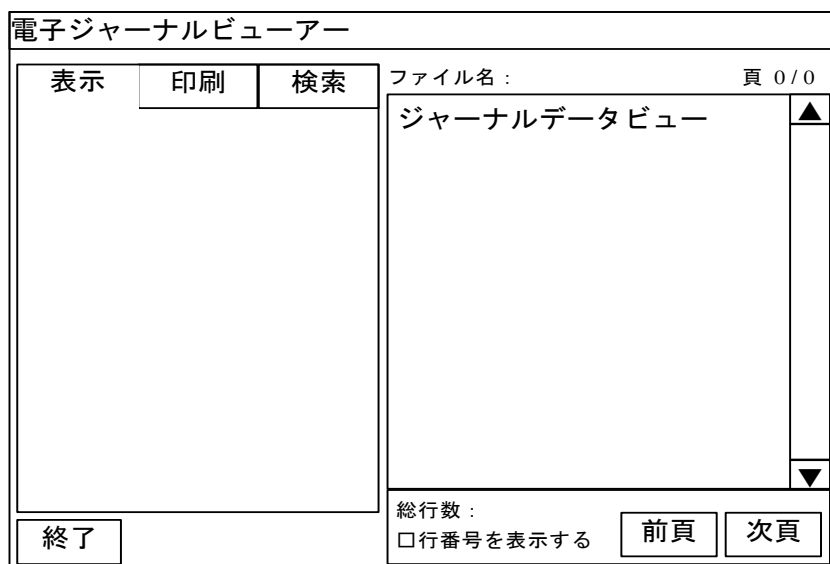
分類	機能概要
ジャーナルファイル/データ表示	ジャーナルファイル一覧表示 ジャーナルフォルダ指定(フォルダボタン押下時) 指定ジャーナルファイルの表示(表示ボタン押下時) データ上下スクロール(スクロールバー配置) データの前後頁移動(前後頁移動ボタン配置) 閲覧データのファイル名表示, 総レコード(行)数表示 レコード番号表示 ON/OFF, 頁位置表示
ジャーナルデータ印刷 (内蔵レシートプリンタへの印刷)	表示されたジャーナルデータに対する印刷 印刷範囲(全行/部分行)指定 印刷の開始および中止
ジャーナルデータ検索	表示されたジャーナルデータに対する検索 検索入力(文字列と数値入力) 検索(現在ポジションを基準とした前後検索)

【拡張機能概要】

機能分類		拡張機能
ジャーナル表示	ファイルパス選択	フォルダ単位での選択が可能 ※
	ジャーナルデータ表示	複数ファイルの連続表示が可能
	頁番号表示	頁番号の表示が追加
	スクロール	ボタンによる頁単位スクロール スクロールバーによる垂直/水平スクロール
ジャーナル印刷	使用プリンタ	設定ファイルによるプリンタ選択
	1 行当たりの文字数	設定ファイルによる指定
	印刷	印刷範囲指定可能
ジャーナル検索	検索	前/次検索が可能
その他	設定ファイルによる動作設定機能	フォント/プリンタ/ジャーナルファイルパス/ウィンドウサイズ/データビュー水平スクロールバー表示非表示の設定が可能

※ 選択したファイルパス下に"EJOURNAL"フォルダが必要です。

【GUI イメージ】



表示タブ	ジャーナルファイル/データ表示画面を表示します。
印刷タブ	印刷画面を表示します。
検索タブ	検索画面を表示します。
ファイル名	ジャーナルデータビューに表示しているファイル名です。
ジャーナルデータビュー	任意のジャーナルファイルデータを表示します。
総行数	スクロールバーおよび前頁/次頁ボタンで表示内容をスクロール可能です。ジャーナルデータビューに表示しているジャーナルデータの総行数の表示。
行番号を表示する	チェックすると、行番号付でジャーナルデータを表示します。
頁(x/y)	表示頁(x)および全頁(y)を表します。
前頁ボタン	ジャーナルデータビューの表示を前頁に移動させます。
次頁ボタン	ジャーナルデータビューの表示を次頁に移動させます。
終了ボタン	電子ジャーナルビューアーを終了します。

【電子ジャーナルビューアーの起動】

EJViewerNET.exe を実行します。なお、EJViewerNET.exe には以下の起動オプションがあります。

- /w:width,height ウィンドウサイズを指定します。width にはウィンドウ幅を、height にはウィンドウ高さを指定します。サイズ 640x480 ピクセルで表示する場合は /w:640,480 とします。
- /d:dir ジャーナルファイルが格納されているディレクトリ(パス)を指定します。デフォルトは "C:¥" です。C:¥ABC を指定する場合は /d:C:¥ABC とします。

オプション省略時は設定ファイルの内容が利用されます。

アプリケーションから起動する場合は次のように行います。

例) VGA(640x480)サイズで起動する場合

```
System.Diagnostics.Process ejViewer = new System.Diagnostics.Process();
ejViewer.StartInfo.FileName = "C:¥¥EJOURNAL¥¥Tools¥¥EJViewerNET.exe";
ejViewer.StartInfo.Arguments = " /w:640,480";
ejViewer.Start();
```

5.2.2. モジュール形態

モジュールタイプ	オブジェクトドライプリプション	プログラマティック ID
.NET EXE	-	-

5.2.3. UI 機能詳細

【表示タブ】

電子ジャーナルビューアー

表示 印刷 検索

ファイル名: 頁 0 / 0

ジャーナルデータビュー

ジャーナルファイルリスト

フォルダ 表示

終了

総行数:
☐ 行番号を表示する

前頁 次頁

印刷タブ	印刷画面を表示します。
検索タブ	検索画面を表示します。
ジャーナルファイルリスト	ジャーナルファイルの一覧を表示します。複数のファイルを選択することができます。
表示ボタン	ジャーナルファイルリストで選択されているファイルの内容をジャーナルデータビューに表示します。
フォルダボタン	ジャーナルフォルダを指定します。
ファイル名	ジャーナルデータビューに表示しているファイル名です。
ジャーナルデータビュー	任意のジャーナルファイルデータを表示します。
総行数	スクロールバーおよび前頁/次頁ボタンで表示内容をスクロール可能です。
行番号を表示する	ジャーナルデータビューに表示しているジャーナルデータの総行数の表示。
頁 (x/y)	チェックすると、行番号付でジャーナルデータを表示します。
前頁ボタン	表示頁 (x) および全頁 (y) を表します。
次頁ボタン	ジャーナルデータビューの表示を前頁に移動させます。
終了ボタン	ジャーナルデータビューの表示を次頁に移動させます。
	電子ジャーナルビューアーを終了します。

表示ボタン押下時、ファイルが選択されていない場合、下記のメッセージボックスを表示します。

ファイルを選択してください。

OK

その他、選択ファイルが壊れている場合等もメッセージボックスを表示します。

【印刷タブ】

設定ファイルに POS プリンタが設定されていないときは、印刷タブ内のコントロールを表示しません。
また、設定されている POS プリンタが見つからない場合は、エラーメッセージボックスを表示し、印刷タブ内のコントロールを表示しません。

表示タブ
検索タブ
印刷範囲

ジャーナルファイル/データ表示画面を表示します。

検索画面を表示します。

印刷する行の範囲を指定します。

全件を選択した場合はジャーナルデータビューで表示している全行が印刷対象となります。

行指定を選択した場合は、開始行と終了行を指定でき、開始行、及び終了行のテキストフィールドをタップしてからジャーナルデータビューの選択行を変えることにより行番号を入力することができます。

プリンタ

印刷に使用するプリンタを指定します。

使用されるプリンタデバイスは設定ファイルに記述します。

印刷/中断ボタン

選択されているジャーナルファイルを印刷/中断します。

ファイル名

ジャーナルデータビューに表示しているファイル名です。

ジャーナルデータビュー

任意のジャーナルファイルデータを表示します。

スクロールバーおよび前頁/次頁ボタンで表示内容をスクロール可能です。

総行数

ジャーナルデータビューに表示しているジャーナルデータの総行数の表示。

行番号を表示する

チェックすると、行番号付でジャーナルデータを表示します。

頁 (x/y)

表示頁 (x) および全頁 (y) を表します。

前頁ボタン

ジャーナルデータビューの表示を前頁に移動させます。

次頁ボタン

ジャーナルデータビューの表示を次頁に移動させます。

終了ボタン

電子ジャーナルビューアーを終了します。

ジャーナルデータビューに表示されているジャーナルファイルのデータを印刷範囲に従って POS プリンタに印刷します。プリンタへの印字は OPOS API により実現します。

用紙幅を超えるデータの場合は複数行に印刷します。レシートに印刷する場合は用紙をカットします。

印刷フォーマット

印刷範囲内でデータファイルが変わるごとにファイル名を印刷します。

=====
ファイル名 : XXXXX

印刷開始行データ
印刷開始 + 1 行目 データ
...
=====
ファイル名 : XXXXX

印刷開始 + n 行目 データ
...
印刷終了行データ
=====

印刷時のエラー処理

印刷時にエラーが発生すると、エラーメッセージボックスを表示します。

エラーには主に以下のようなものがあります。

- ・ジャーナルデータビューにデータを表示していないとき
- ・印刷範囲指定が不正なとき
- ・プリンタが使用中のとき
- ・プリンタカバーが開いているとき
- ・用紙切れのとき
- ・その他

エラーメッセージボックスの表示イメージの例を以下に示します。

ジャーナルデータビューにデータが表示されていない場合

印刷対象データがありません。
OK

プリンタカバーが開いているとき

プリンタカバーが開いています。
再印刷しますか。
はい いいえ

「はい」が押された場合、「プリンタエラーのため印刷を中断しました。」のメッセージを印刷し、設定ファイルの CutMode に設定された動作を行った後、再印刷します。

「いいえ」が押された場合は印刷を中断します。

【検索タブ】

電子ジャーナルビューアー		
表示	印刷	検索
 検索する文字列		ファイル名 : 頁 0 / 0 ジャーナルデータビュー
前を検索 次を検索		
終了		総行数 : ☐ 行番号を表示する 前頁 次頁

表示タブ

ジャーナルファイル/データ表示画面を表示します。

印刷タブ

印刷画面を表示します。

検索する文字列

検索する文字列を入力します。

前を検索ボタン

ジャーナルデータビューで選択されている行より前方向へ文字列を検索します。

次を検索ボタン

ジャーナルデータビューで選択されている行より後ろ方向へ文字列を検索します。

ファイル名

ジャーナルデータビューに表示しているファイルの名前です。

ジャーナルデータビュー

文字列が見つかった場合、その文字列が含まれるジャーナルファイルのレコードを最上行に反転表示します。

総行数

ジャーナルデータビューに表示しているジャーナルデータの総行数を表します。

行番号を表示する

ジャーナルデータビューに表示しているジャーナルデータの総行数の表示。

頁 (x/y)

表示頁 (x) および全頁 (y) を表します。

前頁ボタン

ジャーナルデータビューの表示を前頁に移動させます。

次頁ボタン

ジャーナルデータビューの表示を次頁に移動させます。

終了ボタン

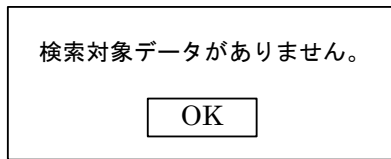
電子ジャーナルビューアーを終了します。

ジャーナルデータビューで選択された行からデータを検索します。検索結果はジャーナルデータビューに反映します。全てのデータを検索し終わった場合、下記のメッセージボックスを表示します。

検索が終了しました。 OK

検索時のエラー処理

エラー発生時にはメッセージボックスを表示します。以下にメッセージボックスの表示イメージ例(ジャーナルデータビューにデータが表示されていない状態での検索)を示します。



【ジャーナルデータビュー表示について】

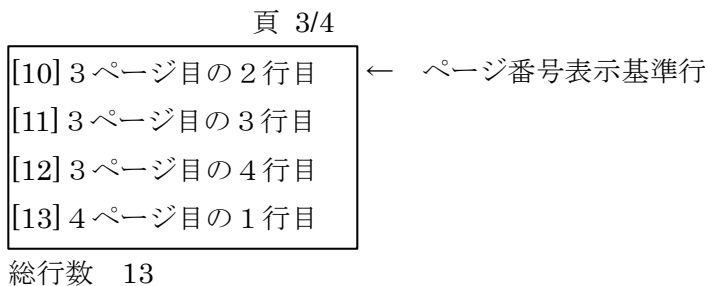
・ページ表示

ページはジャーナルデータビューに一度に表示できる行数分が 1 ページとなります。

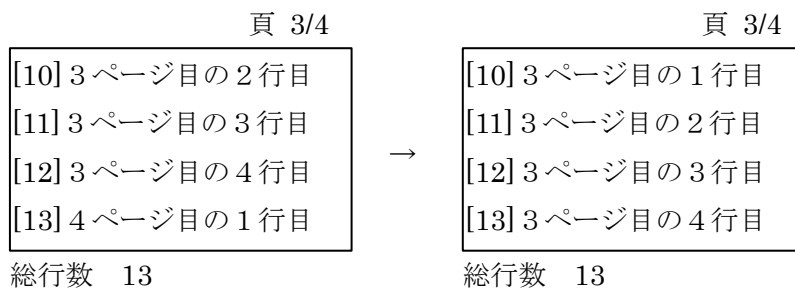
10 行表示ジャーナルデータビューの場合、10 行で 1 ページとなります。

表示の基準はリストの一番上に表示されている行が何ページ目に属するかで表示しています。

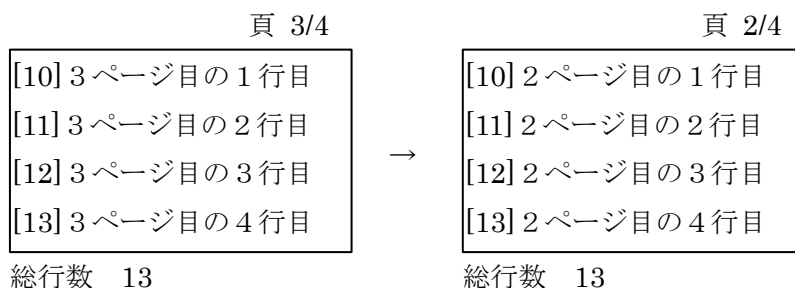
例) ジャーナルデータビューが 4 行表示の場合のイメージ



「前頁」ボタンを押すと、表示しているリストの一番上の行が、属しているページの途中であった場合、そのページの先頭に表示を移動します。



属しているページの先頭行であった場合は、その前のページ先頭行に表示を移動します。



最初のページの先頭行が一番上に表示されるまで、表示移動できます。

「次頁」ボタンを押すと、表示しているリストの一番上の行が属するページの次のページに表示移動します。

[10] 1 ページ目の 3 行目	→	[10] 2 ページ目の 1 行目
[11] 1 ページ目の 4 行目		[11] 2 ページ目の 2 行目
[12] 2 ページ目の 1 行目		[12] 2 ページ目の 3 行目
[13] 2 ページ目の 2 行目		[13] 2 ページ目の 4 行目
総行数 13		総行数 13

最終ページの最後の行が表示される位置まで、表示移動できます。

5.2.4. 設定ファイル(EJViewer.ini)

【概要】

電子ジャーナルビューアー用設定ファイルです。ウィンドウサイズ、ジャーナルフォルダの位置、フォントサイズ、プリンタ等の設定を行うためのファイルです。

【設定ファイル(EJViewer ini)の格納場所】

EJViewer.exe と共通のフォルダに格納されます。

【設定内容】

1. フォント指定

フォントは、ラベル/テキストフィールド/ボタン/リストボックス/ラジオボタン/チェックボックス単位で指定します。

書式 キー名=フォント名,スタイル,フォントサイズ

キー名	対応機能
FontLabel	ラベルのフォントを設定します
FontTextField	テキストフィールドのフォントを設定します
FontButton	ボタンのフォントを設定します
FontListBox	リストボックスのフォントを設定します(プロポーショナルフォント以外)
FontRadioButton	ラジオボタンのフォントを設定します
FontCheckBox	チェックボックスのフォントを設定します

例 FontLabel=MS P ゴシック,N,12

補足 省略時、リストボックスは”MS ゴシック,N,12”、その他は”MS P ゴシック,N,12”に設定されます。リストボックスにプロポーショナルフォントを指定しないでください。また、フォント、フォントサイズによっては表示が乱れますのでご注意ください。

2. プリンタ名指定

ジャーナルプリンタの印刷に使用するプリンタデバイス名を指定します。

書式 PrinterName=プリンタデバイス名

例 PrinterName= SHARPRECPRT58

補足 プリンタデバイス名の指定は必須(省略不可)です。SHARPRECPRT58 等、プリンタに合わせて指定してください。

3. プリンタの一行当たりの文字数指定

ジャーナルプリンタの一行あたりの文字数を指定します。

書式 LineCharCnt=文字数

例 LineCharCnt=36

補足 省略時、36 に設定されます。

4. ウィンドウサイズ

電子ジャーナルビューアーのフレーム(ウィンドウ)サイズを指定します。

書式 WindowWidth=ウィンドウ幅

例 WindowHeight=ウィンドウ高さ

補足 省略時、デスクトップ画面サイズとなります。

5. 電子ジャーナルファイルパス

電子ジャーナルビューアーが最初に電子ジャーナルファイルを検索するパスを指定します。

書式 EJFilePath=電子ジャーナルファイル保存パス

例 EJFilePath=C:¥

補足 省略時 “C:¥”となります。

6. レシートカットモード

印刷後のカット、スタンプ印刷の動作を指定します。

書式 CutMode=カットモード

カットモードの指定は以下のようになっています。

カット後スタンプを印刷 CUTSTAMP

カットを行う CUT

カットを行わない NOCUT

例 CutMode=CUTSTAMP

補足 省略時は CUTSTAMP です。

7. ジャーナルデータビュー水平スクロールバー表示モード

ジャーナルデータビューに水平スクロールバーを表示するかを設定することができます。

書式 HScrollBar=表示モード

表示モード指定は以下のようになっています。

水平スクロールバーを表示する ON

水平スクロールバーを表示しない OFF

例 HScrollBar =OFF

補足 省略時は OFF です。

【設定ファイル例(出荷時設定)】

#ラベルフォント設定

FontLabel=MS P ゴシック,N,12

#テキストフィールドフォント設定

FontTextField=MS P ゴシック,N,12

#ボタンフォント設定

FontButton=MS P ゴシック,N,12

#リストボックスフォント設定

FontListBox=MS ゴシック,N,12

#ラジオボタンフォント設定

FontRadioButton=MS P ゴシック,N,12

#チェックボックスフォント設定

FontCheckBox=MS P ゴシック,N,12

#プリンタ名

PrinterName=SHARPRECPRT58

#一行あたりの文字数

LineCharCnt=

#ウィンドウ幅

WindowWidth=

#ウィンドウ高さ

WindowHeight=

#ジャーナルファイルパス

EJFilePath=

#レシートカットモード

CutMode=

#ジャーナルデータビュー水平スクロールバーON/OFF

HScrollBar=

(注意)

- 先頭#で始まる行は、コメントとして扱われます。
- 指定文字列(キー,データ)中にスペース(空白文字)を入れることは出来ません。

5.2.5. 特記事項

- EJViewerNet.exe は、C:¥EJOURNAL¥TOOLS に格納されます。
- システム設定-ディスプレイのカスタマイズの、「テキスト、アプリ、その他の項目のサイズを変更する:」設定は、「100%(推奨)」にのみ、対応しています。
- 電子ジャーナルファイルが格納されているフォルダは、指定したディレクトリ+”¥EJOURNAL”となります。(”C:¥ABC”を指定した場合は” C:¥ABC¥EJOURNAL”となります。)
- エスケープシーケンスには対応しておりません。
ジャーナルファイルのデータ中に 20h 以下のコードが含まれている場合、それらのコードは 20h(スペース)に置き換えて表示/印刷します

5.3. 電子ジャーナルファイルチェック (ActiveX DLL)

5.3.1. 概要

電子ジャーナルファイルの診断、修復を行います。

【診断・修復シーケンス】

1. 指定されたジャーナルファイルのヘッダ情報を読み込み、ジャーナルファイル識別子をチェックします。
ジャーナルファイルでない場合は、エラーを戻します。
2. ジャーナルファイルのヘッダとデータのチェックを行い、以下の場合はエラーを戻します。
 - ・ヘッダに登録されているレコードサイズが 0 以下の場合
 - ・ジャーナルファイルサイズがジャーナルファイルヘッダサイズより小さい場合
3. ヘッダに登録されているレコードサイズとファイルサイズの整合性チェックを行います。
不整合の場合はファイルサイズが(ヘッダサイズ + レコードサイズ * n)になるように、0x00 を追加して、ジャーナルファイルを修復(元のジャーナルファイルは別名で保存)します。

5.3.2. オブジェクト形態

オブジェクトタイプ	オブジェクトドライブリプション	プログラマティック ID
ActiveX DLL	SHARP EJFILECHECK	SHARPEJFC.EJFileCheck

5.3.3. 共通 I/F サマリー

メソッド名称	ドライブリプション
EJFileChk	電子ジャーナルファイルの診断・修復を行います

5.3.4. メソッド詳細

EJFileChk メソッド

形式	[VC#]	Int32 EJFileChk(String Path, String FileName, out String BackupFileName)
	[VB.NET]	Function EJFileChk(ByVal Path As String, ByVal FileName As String, ByRef BackupFileName As String) As Int32

パラメーター	説明
<i>Path</i>	診断するジャーナルファイルのパス
<i>FileName</i>	診断するジャーナルファイル名
<i>BackupFileName</i>	バックアップファイル名が戻ります。 (ジャーナルファイル修復時のみ) バックアップファイルは、「ファイル名.日付」となります。 例えば、2006/4/1 に ErrEJFile.ejd を修復した場合、 「ErrEJFile.ejd.20060401」名でバックアップされます。

説明	ジャーナルファイルを診断し、ジャーナルファイル異常を検出した場合は修復します。 ジャーナルファイル修復時は、元のジャーナルファイルを <i>BackupFileName</i> でバックアップします。電子ジャーナルファイルのパスは <i>Path</i>+“¥EJOURNAL”となります。 (<i>Path</i> に“C:¥ABC”を指定した場合は“ C:¥ABC¥EJOURNAL”となります。)
----	--

戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	指定ジャーナルファイルが正常か、修復に成功しました
PFO_Illegal_Error	パラメーター不正
PFO_Device_Error	デバイス(またはメディア)が存在しない
PFO_Exist_Error	指定ファイルがオープンされていた場合
PFO_NoExist_Error	指定ファイル未存在
PFO_FileProtect_Error	指定ファイルにアクセスできない、リネーム失敗等
PFO_FileFormat_Error	ジャーナルファイルでない、ヘッダ情報が不正な場合等
PFO_FileRead_Error	ジャーナルファイル読み込み失敗
PFO_FileWrite_Error	ジャーナルファイル書き込み失敗、バックアップ失敗時等

第6章 デバイス支援

6.1. 液晶制御 (ActiveX DLL)

6.1.1. 概要

LCD のバックライト制御を行うメソッドです。本メソッドは POS 環境でのみ利用できます。

6.1.2. オブジェクト形態

オブジェクトタイプ	オブジェクトドライプリプション	プログラマティック ID
ActiveX DLL	SHARP DISPLAY CONTROL	SHARP.DISPLAY

6.1.3. 共通 I/F サマリー

メソッド名称	ドライプリプション
SetBackLight	内蔵ディスプレイのバックライト ON/OFF を行います
SetBackLightBrightness	内蔵ディスプレイのバックライト輝度を設定します

6.1.4. メソッド詳細

SetBackLight メソッド

形式 [VC#] Int32 SetBackLight(Int32 *LcdControl*)

[VB.NET] Function SetBackLight(ByVal *LcdControl* As Int32) As Int32

パラメーター	説明
--------	----

<i>LcdControl</i>	LCD バックライト ON/OFF 指定 PFO_DL_BacklightOn : バックライト ON(点灯) PFO_DL_BacklightOff : バックライト OFF(消灯)
-------------------	--

説明 LCD バックライトの状態を設定します。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

PFO_Success	正常終了
PFO_Illegal_Error	パラメーターエラーです
PFO_Device_Error	デバイスエラーです

SetBackLightBrightness メソッド

形式 [VC#] Int32 SetBackLightBrightness(Int32 *NewBrightness*)

[VB.NET] Function SetBackLightBrightness(ByVal *Brightness* As Int32) As Int32

パラメーター	説明
--------	----

<i>NewBrightness</i>	LCD バックライト輝度指定 1～100 輝度
----------------------	----------------------------

説明 LCD バックライトの輝度を設定します。

戻り値 次の値のいずれかが戻されます。

値	意味
---	----

PFO_Success	正常終了
PFO_Illegal_Error	パラメーターエラーです
PFO_Device_Error	デバイスエラーです

6.1.5. 特記事項

- ・バックライトの輝度は、バックライトの ON(点灯)／OFF(消灯)状態に関係なく設定可能です。また、バックライトを OFF(消灯)してもバックライトの輝度値は保持されます。例えば、バックライトの輝度が 50

に設定されている状態で、バックライトを **OFF** (消灯) から **ON** (点灯) にするとバックライトの輝度は **50** で **ON** (点灯) されます。

- バックライト **OFF** (消灯) から **ON** (点灯) への移行は、バックライトの **ON** (点灯) 命令を実行するか、外部入力 (キー、マウスまたはタッチパネル) 操作を行うことで **ON** (点灯) になります。
- バックライトの輝度は、**1**～**100** の範囲で指定しますが、内部的には **16** 段階で輝度設定されます。
- **AC** 電源供給が無く内蔵バッテリーによる動作中、バックライトの輝度設定は **1** 以外に変更できません。

6.2. POS プリンタ外字 (DLL)

6.2.1. 概要

POS プリンタ用外字を取得します。本関数は POS 環境でのみ利用できます。

6.2.2. 共通 I/F サマリー

関数名称	ドライブリプション
GetGaijiData	POS プリンタ用外字取得

6.2.3. 関数詳細

GetGaijiData 関数

形式 **[VC#]** `[DllImport("POSUTIL.dll")]`
 `Int32 GetGaijiData(String filename, UInt16 fonttype,`
 `UInt16 gcode, ref Byte databuf)`
[VB.NET] `<DllImport("POSUTIL.dll")> _`
 `Private Shared Function GetGaijiData(`
 `ByVal filename As String, ByVal fonttype As UInt16,`
 `ByVal gcode As UInt16, ByRef databuf As Byte)As Int32`

パラメーター	説明
<i>filename</i>	外字ファイル名を指定します。 ファイル名は、フルパスで指定してください。 ファイル拡張子は、.GFU です。
<i>fonttype</i>	0 : 24 ドット×24 ドット 1 : 16 ドット×16 ドット
<i>Gcode</i>	外字コードを指定します。(7721H~777EH の範囲内)
<i>Databuf</i>	外字データ格納バッファを指定します。 (24 ドット:72 バイト、16 ドット:32 バイト必要)

説明 内蔵レシートプリンタ外字作成ユーティリティで作成した外字フォントデータを外字ファイル (.GFU) から読み込みます。データの取り出しのみを行います。

戻り値 次の値のいずれかが戻されます。

値	意味
0	正常終了
1	パラメーターエラー
2	ファイルオープンエラー
3	ファイルリードエラー
4	外字データエラー
5	外字データ未登録
以外	その他のエラー

6.3. POS キーボード制御 (DLL)

6.3.1. 概要

POS キーボードマトリックスの制御を行います。本関数は POS 環境でのみ利用できます。

6.3.2. 共通 I/F サマリー

関数名称	ドライブリプション
------	-----------

PosKdown	POS キーボードマトリックス 取得、設定
----------	-----------------------

6.3.3. 関数詳細

PosKdown 関数

形式	[VC#]	[DllImport("POSUTIL.dll")] Int32 PosKdown (Int32 mode, ref Byte keytype, String filename, ref Byte pblk)
	[VB.NET]	<DllImport("POSUTIL.dll")> _ Private Shared Function PosKdown (ByVal mode As Int32, ByRef keytype As Byte, ByVal filename As String, ByRef pblk As Byte)As Int32

形式

パラメーター	説明
Mode	0 : 取得 1 : 設定 (ファイルから設定) 2 : 設定 (キー配列から設定)
Keytype	mode が 0 のとき、接続されているキーボードの種類を格納するエリアへのアドレス。 mode が 1 または 2 のとき、キー配列を設定する対象のキーボードの種類を指定したデータへのアドレス。
Filename	mode が 0 または 2 のときはダミーです。 mode が 1 のとき、ロードするキーマトリックスファイル名を指定します。 拡張子(.TXT)は省略不可です。フルパス名で指定してください。
Pblk	mode が 0 のとき、キー配列を格納するエリアへのアドレス。 mode が 2 のとき、設定するキー配列へのアドレス。

説明 テキストエディタで作成したキーマトリックスファイル(.TXT)から設定コードを読み込み、POS キーに登録します。また、現在設定されているコードを取得します。

戻り値 次の値のいずれかが戻されます。

値	意味
0	正常終了
1	パラメーターエラー
2	ファイルオープンエラー (ファイルからの設定時)
3	ファイルリードエラー (ファイルからの設定時)
4	キーコードデータが間違っています (設定時)
5	キーコード設定エラー (設定時)
6	指定キーボードの種類と接続されているキーボードが不一致 (設定時)
以外	その他のエラー

【keytype の値】

mode が 0 の場合、現在接続されている POS キーボードの種類が格納されて戻ります。mode が 1 の場合、キー配列を設定する対象の POS キーボードの種類を指定します。

mode が 0 (取得)	0	POS キーボード接続されていません
	1	42 キー (RZ-4PK1) の POS キーボードが接続されています
	2	予約
mode が 1 (設定)	1	42 キー (RZ-4PK1) の POS キーボード配列を設定
	2	予約(使用しない)

【pblk の構造】

pblk は、バイナリデータ (BYTE 型) で格納されています。

1 キー番号に対して 4 バイトのデータが格納されています。(合計 168 バイト)

複合キーが設定されていない場合 1 バイトにキーのコード、残りの 3 バイトに 0 が格納されます。

複合キーが設定されている場合、2 バイト～4 バイトにキーのコードが格納されます。

構造は次に示すとおりです。

pblk の先頭	+0	01	キー番号 1 のコード
	+1	00	キー番号 1 のコード
	+2	00	キー番号 1 のコード
	+3	00	キー番号 1 のコード
	+4	02	キー番号 2 のコード
	+5	00	キー番号 2 のコード
		.	
		.	
	+166	00	キー番号 42 のコード
	+167	00	キー番号 42 のコード

【補足説明】

キーマトリックスファイル、キーコードについては、「POS ユーティリティ説明書」を参照してください。

第7章 システム支援

7.1. システム起動 (DLL)

7.1.1. 概要

システム起動時処理を行う関数です。本関数は POS 環境でのみ利用できます。
システム起動、停止についての詳細説明は、「基本ソフトウェア説明書」を参照ください。

7.1.2. 共通 I/F サマリー

関数名称	ドライブリプション
WaitPosapl	システム起動処理の実行

7.1.3. 関数詳細

WaitPosapl 関数

形式	[VC#]	[DllImport("POSSYS32.dll")] Int32 WaitPosapl(Int16 swch)
	[VB.NET]	<DllImport("POSSYS32.dll")> _ Private Shared Function WaitPosapl(ByVal swch As Int16)As Int32

パラメーター	説明
Swch	0 を指定します。(電源 ON 時に起動プロセスの処理完了を待つ)

説明 最初のアプリケーションの起動の先頭で呼び出します。
BOOTPOS.INI の [boot] セクション内 run エントリで指定された起動時実行プログラムの実行、
[powerfail] セクション内 run エントリで指定された停電発生時実行プログラムを登録します。

戻り値 次の値のいずれかが戻されます。

値	意味
0	正常終了
2, 7, 10, 11, 12, 14, 15, 16, 17, 21, 96, 98	予約
3	スタンプロゴ設定エラー
4	16×16 外字設定エラー
5	24×24 外字設定エラー
6	キーボードマトリックス設定エラー
8	RUN エラー
13	COM 電源供給エラー
20	停電監視プロセス起動エラー
97	ドライバが存在しない
99	その他のエラー

7.2. システム停止 (DLL)

7.2.1. 概要

POS システムの終了処理を行う関数です。本関数は POS 環境でのみ利用できます。
システム起動、停止についての詳細説明は、「基本ソフトウェア説明書」を参照ください。

7.2.2. 共通 I/F サマリー

関数名称	ドライブリプション
POS_EXIT	システム停止処理

7.2.3. 関数詳細

POS_EXIT 関数

形式	[VC#]	[DllImport("POSSYS32.dll")] Int32 POS_EXIT(Int32 <i>flag</i>)
	[VB.NET]	<DllImport("POSSYS32.dll")> _ Private Shared Function POS_EXIT(ByVal <i>flag</i> As Int32) As Int32

パラメーター	説明
<i>flag</i>	シャットダウン動作を指定 0 : 再起動 1 : 電源 OFF

説明 本関数を呼び出す前に他のアプリケーションを終了しておく必要があります。
本関数終了後、電源 OFF または再起動されます。

戻り値 次の値のいずれかが戻されます。

値	意味
0	正常終了
1	パラメーターエラー
2	異常終了

※ 正常終了した場合、システムが終了されるため戻り値を取得できません。

7.3. ミラーリング監視 (DLL)

7.3.1. 概要

POS アプリケーションから、ミラーリングの動作状態を監視する関数です。

本関数は POS 環境でのみ利用できます。

ミラーリングについての詳細説明は、「ミラーリング説明書」を参照ください。

RZ-A476S のみミラーリングを構成することが可能です。RZ-A396S はミラーリングを構成できません。

7.3.2. 共通 I/F サマリー

関数名	ドライブリプション
GetMirrorMode	ミラー/ 単体モードの取得
GetNumDisks	接続されているドライブの台数
GetMirrorStatus	ミラー状態の取得
GetPrimaryDiskStatus	ドライブ1(Port4)の状態の取得
GetSecondaryDiskStatus	ドライブ2(Port5)の状態の取得

7.3.3. 関数詳細

GetMirrorMode 関数

```
形式    [VC#]    [DllImport("POSMIRROR.dll")]
          Int32 GetMirrorMode( )
          [VB.NET] <DllImport("POSMIRROR.dll")> _
                  Private Shared Function GetMirrorMode( ) As Int32
```

説明 現在の動作モードを取得します。

戻り値 次の値のいずれかが戻されます。

値	意味
0	情報取得エラー
1	単体モード
2	ミラーモード

GetNumDisks 関数

```
形式    [VC#]    [DllImport("POSMIRROR.dll")]
          Int32 GetNumDisks( )
          [VB.NET] <DllImport("POSMIRROR.dll")> _
                  Private Shared Function GetNumDisks( ) As Int32
```

説明 接続されているドライブの台数を取得します。

戻り値 次の値のいずれかが戻されます。

値	意味
0	情報取得エラー
1～	接続されているドライブの台数

GetMirrorStatus 関数

形式 [VC#] [DllImport("POSMIRROR.dll")]
Int32 GetMirrorStatus()
[VB.NET] <DllImport("POSMIRROR.dll")> _
Private Shared Function GetMirrorStatus() As Int32

説明 現在のミラーリング動作状態を取得します。
RZ-A476S では、正常時 1（単体モード時）か 2（ミラーモード時）の状態が返されます。
構築中は 4 が返されますが、戻り値が 1、2、4 以外のときは、ミラー動作エラーです。

ドライブ毎の状態を調べるには、**GetPrimaryDiskStatus**、
GetSecondaryDiskStatus 関数を使用してください。これらの関数では、正常動作して
いるドライブに対する戻り値は 2（正常）となり、異常が発生しているドライブを特定できます。

戻り値 次の値のいずれかが戻されます。

値	意味
0	情報取得エラー 情報が取得できなかった（ドライブが接続されていない、認識していない）
1	ミラーモード以外（単体モード） 単体モードで動作（ミラー設定していない）
2	正常（ミラーモード） ドライブ 2 台認識しており、ミラーリングが行われている
3	ミラー機能停止中 ドライブ1/2いずれかもしくは両方のドライブにミラー設定してあるが、ミラーリングはされていない
4	構築中（ミラーモード） ミラー設定があり構築を行っている状態
5	その他の状態 不明な値が status として返って来ている

GetPrimaryDiskStatus 関数

形式 [VC#] [DllImport("POSMIRROR.dll")]
Int32 GetPrimaryDiskStatus()
[VB.NET] <DllImport("POSMIRROR.dll")> _
Private Shared Function GetPrimaryDiskStatus() As Int32

説明 ドライブ1の動作状態を取得します。
RZ-A476S では、正常時 1（単体モード時）か 2（ミラーモード時）の状態が返ります。
構築中は 4 が返されますが、戻り値が 1、2、4 以外のときは、ミラー動作エラーです。

戻り値 次の値のいずれかが戻されます。

値	意味
0	情報取得エラー 情報が取得できなかった（ドライブが接続されていない、認識していない）
1	ミラーモード以外（単体モード） 単体モードで動作（ミラー設定していない）
2	正常（ミラーモード） ドライブ 2 台認識しており、ミラーリングが行われている
3	ミラー機能停止中 ドライブ1/2いずれかもしくは両方のドライブにミラー設定してあるが、ミラーリングはされていない
4	構築中（ミラーモード） ミラー設定があり構築を行っている状態
5	その他の状態 不明な値が status として返って来ている

GetSecondaryDiskStatus 関数

形式 [VC#] [DllImport("POSMIRROR.dll")]
 Int32 GetSecondaryDiskStatus()
 [VB.NET] <DllImport("POSMIRROR.dll")> _
 Private Shared Function GetSecondaryDiskStatus() As Int32

説明 ドライブ2の動作状態を取得します。
RZ-A476S では、正常時 1（単体モード時）か 2（ミラーモード時）の状態が返ります。
構築中は 4 が返されますが、戻り値が 1、2、4 以外のときは、ミラー動作エラーです。

戻り値 次の値のいずれかが戻されます。

値	意味	
0	情報取得エラー	情報が取得できなかった(ドライブが接続されていない、認識していない)
1	ミラーモード以外（単体モード）	単体モードで動作(ミラー設定していない)
2	正常（ミラーモード）	ドライブ 2 台認識しており、ミラーリングが行われている
3	ミラー機能停止中	ドライブ1/2いずれかもしくは両方のドライブにミラー設定してあるが、ミラーリングはされていない
4	構築中（ミラーモード）	ミラー設定があり構築を行っている状態
5	その他の状態	不明な値が status として返って来ている

GetDiskId 関数

形式 [VC#] [DllImport("POSMIRROR.dll")]
 bool GetDiskId(Int32 disk, ref Byte databuf)
 [VB.NET] <DllImport("POSMIRROR.dll")> _
 Private Shared Function GetDiskId(ByVal disk As Int32, ByRef databuf As Byte) As bool

パラメーター	説明
<i>disk</i>	0 : Port 0 のドライブ 1 : Port 1 のドライブ
<i>databuf</i>	ドライブの ID 情報格納バッファを指定します。 (長さは 512 バイト)

説明 *disk* パラメーターで指定したドライブの ID 情報 (IDENTIFY DEVICE) を取得します。
 512 バイトの ID 情報は *databuf* パラメーターで指定したバッファに保存されます。

戻り値 次の値のいずれかが戻されます。

値	意味
true	正常終了
false	情報取得エラー

【補足説明】

ドライブの IDENTIFY DEVICE コマンド (ドライブの ATA 規格で定められている、デバイス情報取得コマンド) で取得出来る 512 バイトのデータを取得します。

ドライブの詳細な情報を取得したい場合、本関数を使用してください。

RZ-A476S/A396S のドライブでは、下記情報が取得できます。

バイト位置	属性	意味	
0-19	-	非公開	
20-39	文字列	ドライブシリアル番号 ASCII コード 20 キャラクタ	
40-45	-	非公開	
46-53	文字列	ファームウェア版数 ASCII コード 8 キャラクタ	
54-93	文字列	ドライブモデル名 ASCII コード 40 キャラクタ左詰め	
94-119	-	非公開	
120-123	数値	ドライブの総セクタ数 (LBA モード)	※1
124-511	-	非公開	

※1 ミラーリングの場合、実際にユーザーがアクセス可能なセクタ数は本セクタ数より小さくなります。

属性が数値のデータの場合、最下位バイトから順に保存されます。

例:

バイト位置 [2-3] 論理シリンダが 0x3FFFF の場合

→ バイト位置 2 = 0xFF、3 = 0x3F

バイト位置 [120-123] 総セクタ数が 0x04A85300 の場合

→ バイト位置 120 = 0x00、121 = 0x53、122 = 0xA8、123 = 0x04

属性が文字列のデータの場合、WORD 単位で返されるため、WORD 単位での入れ替えを行います。

例:

バイト位置[20-39] シリアル番号が、"17FTH23-0118 N00 064" の場合

バイト位置 = ASCII 文字 (Hex 値)

20 = '7' (0x37)	21 = '1' (0x31)	22 = 'T' (0x54)	23 = 'F' (0x46)
24 = '2' (0x32)	25 = 'H' (0x48)	26 = '-' (0x2D)	27 = '3' (0x33)
28 = '1' (0x31)	29 = '0' (0x30)	30 = '8' (0x38)	31 = '1' (0x31)
32 = 'N' (0x4E)	33 = ' ' (0x20)	34 = '0' (0x30)	35 = '0' (0x30)
36 = '0' (0x30)	37 = ' ' (0x20)	38 = '4' (0x34)	39 = '6' (0x36)

.NET で使用できるデータに変換するには、以下のサンプルプログラムを参考にしてください。

サンプルプログラム:

```
Byte[] buffer = new Byte[512];      //ドライブ情報取得用バッファ
Byte flipbyte;                      //文字列配置入れ替え用バッファ
string diskserial;                  //Port 0 のシリアル番号
UInt16 numcylinder;                 //Port 0 の論理シリンダ数
UInt32 numsectors;                  //Port 0 のユーザーアクセス可能な総セクタ数

if (GetDiskId (0, ref buffer[0]))//Port 0 のドライブ情報を取得する
{
    //取得した情報からドライブのシリアル番号(ASCII)を.NET の string に変換する。
    //文字列も WORD 配置となっているので、先に配置を入れ替えておく。
    for (int i = 20; i < 40; i += 2)
    {
        flipbyte = buffer[i];
        buffer[i] = buffer[i + 1];
        buffer[i + 1] = flipbyte;
    }

    //ASCII から UTF8 に変換する。
    diskserial = System.Text.Encoding.UTF8.GetString(buffer,20,20);

    //取得した情報からドライブの論理シリンダ数を変換する。
    numcylinder = BitConverter.ToUInt16(buffer, 2);

    //取得した情報からドライブの総セクタ数を変換する。
    numsectors = BitConverter.ToUInt32(buffer, 120);
}
```

GetSMARTInfo 関数

形式 [VC#] [DllImport("POSMIRROR.dll")]
 bool GetSMARTInfo(Int32 disk, ref Byte databuf)
 [VB.NET] <DllImport("POSMIRROR.dll")> _
 Private Shared Function GetSMARTInfo(ByVal disk As Int32,
 ByRef databuf As Byte) As bool

パラメーター	説明
<i>disk</i>	0 : Port 0 のドライブ 1 : Port 1 のドライブ
<i>databuf</i>	ドライブの SMART 情報格納バッファを指定します。 (長さは 256 バイト)

説明 *disk* パラメーターで指定したドライブの SMART 情報を取得します。
 256 バイトの SMART 情報は *databuf* パラメーターで指定したバッファに保存されます。

戻り値 次の値のいずれかが戻されます。

値	意味
true	正常終了
false	情報取得エラー

【補足説明】

ドライブの SMART 情報を取得します。

SSD の SMART 情報取得は、GetSMARTAttribute 関数、GetSMARTThreshold 関数を使用します。

本関数では、SMART 属性情報と SMART 閾値情報から総合結果を判定して返します。

データには、判定結果(現在の属性値が閾値以下の属性数)、各属性の情報が返されます。

総合結果が 1 以上の場合、障害が発生する危険性が高いと判断されるため、警告等の表示、ドライブ交換を実施してください。

現在の属性値と過去最悪の属性値はドライブにより自動的に更新されます。

属性値の範囲はドライブメーカーが規定しているため、属性により異なります。

閾値はドライブメーカーにより設定されており、閾値が 0(設定されていない)の属性もあります。

閾値が 0 の場合、判定結果は常に 0 となります。

バイト位置	情報		値
0	総合結果		0 : 閾値以下の属性は無い 1 以上 : 閾値以下の属性の数
1	SMART 属性の個数		0~30 : 本データに含まれる属性の数
2	属性 1	属性 ID	属性 ID(下表参照)
3		現在の属性値	値は属性により異なる
4		過去最悪の属性値	値は属性により異なる
5		閾値	ドライブメーカーにより属性毎に設定
6		判定結果	0 : 現在の属性値が正常範囲内 1 : 現在の属性値が閾値以下
7-151	属性 2~ 属性 30	形式は属性 1 と同じ (バイト 2~6 を参照)	バイト 1 で示される SMART 属性の個数分、 属性値が返される
152-255	予約		未使用

ドライブに保存されている SMART 属性情報と閾値情報を取得したい場合、GetSMARTAttribute 関数
 及び GetSMARTThreshold 関数を使用してください。

GetSMARTAttribute 関数

形式 [VC#] [DllImport("POSMIRROR.dll")]
 bool GetSMARTAttribute(Int32 disk, ref Byte databuf)
 [VB.NET] <DllImport("POSMIRROR.dll")> _
 Private Shared Function GetSMARTAttribute(ByVal disk As
 Int32,
 ByRef databuf As Byte) As bool

パラメーター	説明
<i>disk</i>	0 : Port 0 のドライブ 1 : Port 1 のドライブ
<i>databuf</i>	ドライブの SMART 属性情報格納バッファを指定します。 (長さは 512 バイト)

説明 *disk* パラメーターで指定したドライブの SMART 属性情報を取得します。
 512 バイトの SMART 属性情報は *databuf* パラメーターで指定したバッファに保存されます。

戻り値 次の値のいずれかが戻されます。

値	意味
true	正常終了
false	情報取得エラー

【補足説明】

ドライブ (SSD) の SMART 属性情報を取得します。

SMART 属性情報は 512 バイトのデータで構成され、以下の形式になっています。

バイト位置	意味	
0-1	SMART のデータ形式版数	
2	属性 1	属性 ID
3-4		非公開
5		現在の属性値
6		過去最悪時の属性値
7-12		拡張属性値
13		非公開
14-361	属性 2～ 属性 30	属性 2～30 の形式は、属性 1 (バイト 2～13) と同じ
362-510	非公開	
511	チェックサム (先頭から 511 バイト分のデータを加算した時の下位 1 バイトに対する 2 の補数)	

属性 1～属性 30 に、属性の ID と、その属性の現在の属性値及び過去最悪時の属性値が含まれます。
 各属性の閾値は、**GetSMARTThreshold** 関数を使用し、取得します。

属性 ID については、**GetSMARTInfo** 関数の補足説明を参照ください。未使用の属性については、属性 ID が 0 の場合、その属性情報は未使用です。

各属性の閾値と現在の属性値を比較し、現在の属性値が閾値以下の場合、障害が発生する危険性が高いと判断されるため、警告等の表示、ドライブ交換を実施してください。

GetSMARTThreshold 関数

形式 [VC#] [DllImport("POSMIRROR.dll")]
 bool GetSMARTThreshold(Int32 disk, ref Byte databuf)
 [VB.NET] <DllImport("POSMIRROR.dll")> _
 Private Shared Function GetSMARTThreshold (ByVal disk As
 Int32,
 ByRef databuf As Byte) As bool

パラメーター	説明
disk	0 : ドライブ1(Port 4 のドライブ) 1 : Port 5 のドライブ
databuf	ドライブの SMART 閾値情報格納バッファを指定します。 (長さは 512 バイト)

説明 disk パラメーターで指定したドライブの SMART 閾値情報を取得します。
 512 バイトの SMART 閾値情報は databuf パラメーターで指定したバッファに保存されます。

戻り値 次の値のいずれかが戻されます。

値	意味
true	正常終了
false	情報取得エラー

【補足説明】

ドライブ(SSD)の SMART 閾値情報を取得します。

SMART 閾値情報は 512 バイトのデータで構成され、以下の形式になっています。

閾値はドライブメーカーが設定しています。

閾値が 0 の属性は、閾値が設定されていない属性になります。

バイト位置	意味	
0-1	SMART のデータ形式版数	
2	属性 1	属性 ID
3		閾値
4-13		非公開
14-361	属性 2～ 属性 30	属性 2～30 の形式は、属性 1 のバイト 2～13 と同じ
362-510	非公開	
511	チェックサム (先頭から 511 バイト分のデータを加算した時の下位 1 バイトに対する 2 の補数)	

8.2. 内蔵バッテリー情報 (DLL)

8.2.1. 概要

停電時用の内蔵バッテリー情報を取得します。本関数は POS 環境でのみ利用できます。

8.2.2. 共通 I/F サマリー

関数名称	ドライブリプション
GetBatteryInfo	バッテリー情報の取得

8.2.3. 関数詳細

GetBatteryInfo 関数

形式 [VC#] [DllImport("POSSYS32.dll")]
Int32 GetBatteryInfo(ref Byte batteryinfo)
[VB.NET] <DllImport("POSSYS32.dll")> _
Private Shared Function GetBatteryInfo
(ByRef batteryinfo As Byte) As Int32

パラメーター	説明
<i>batteryinfo</i>	内蔵バッテリー情報格納バッファを指定します。 (長さは 256 バイト)

説明 内蔵バッテリー情報を取得します。
バッテリー情報は *batteryinfo* パラメーターで指定したバッファに保存されます。

戻り値 次の値のいずれかが戻されます。

値	意味
0	正常終了
1	パラメーターエラー
2	デバイスエラー

【補足説明】

内蔵バッテリーの情報は、*batteryinfo* パラメーターに以下の形式で保存されます。

バイト位置	情報	値 [単位]
0～1	AtRate 値でバッテリーを充電する場合の残り(フル充電までの)時間の予測値	0～65,535 [min]
2～3	AtRate 値でバッテリーを消費する場合の残り(Empty までの)時間の予測値	0～65,535 [min]
4～5	電池パック内部の温度	0～65,535 [0.1K]
6～7	電池セルの電圧	0～20,000 [mV]
8～9	バッテリーの端子を介して供給される電流	-32,768～32,768 [mAh]
10～11	バッテリーの残容量(予測値)	0～65,535 [mAh]
12～13	フル充電された際の容量(予測値)	0～65,535 [mAh]
14～15	現在の放電レートでの電池の持ち時間	0～65,535 [min]
16～17	充電に必要な電流レート	0～65,535 [mA]
18～19	充電に必要な電圧値	0～65,535 [mV]
20～21	バッテリーのステータス(アラームを含む)	バイナリー
22～23	充電/放電サイクルの回数	0～65,535
24～25	電池パックが製造された日付	バイナリー
26～27	シリアルナンバー	0～65,535
28～47	製造者名	文字列 (20BYTE) 前詰めで終端に NULL
48～67	デバイス名	文字列 (20BYTE) 前詰めで終端に NULL

68～77	バッテリーの化学的性質を含む文字列	文字列(20BYTE) 前詰めで終端に NULL
78～97	製造データ	バイナリー
98～127	バッテリーが信頼できる物であることを証明する文字列	文字列(20BYTE) 前詰めで終端に NULL
128～129	予測残バッテリー容量	0～100 [%]
130～255	予約	未使用

8.3. POS ハードウェア情報(DLL)

8.3.1. 概要

POS ハードウェアの情報を取得します。
本関数は POS 環境でのみ利用できます。

8.3.2. 共通 I/F サマリー

関数名称	ドライブリプション
------	-----------

GetPOSHwInfo	POS ハードウェア情報の取得
--------------	-----------------

8.3.3. 関数詳細

GetPOSHwInfo 関数

形式 [VC#] [DllImport("POSSYS32.dll")]
Int32 GetPOSHwInfo(ref Byte databuf)
[VB.NET] <DllImport("POSSYS32.dll")> _
Private Shared Function GetPOSHwInfo
(ByRef databuf As Byte) As Int32

パラメーター	説明
--------	----

databuf	POS ハードウェア情報格納バッファを指定します。 (長さは 256 バイト)
---------	--

説明 POS ハードウェア情報を取得します。
情報は databuf パラメーターで指定したバッファに保存されます。

戻り値 次の値が戻されます。

値	意味
0	(固定)

【補足説明】

POS ハードウェアの情報を取得します。
情報は databuf パラメーターに以下の形式で保存されます。

バイト位置	情報	値
0～19	機種情報	文字列 最大 20BYTE、空き部 NULL
20～21	CPU 温度センサ値	-32768～32767 温度、符号付き整数値
22～23	基板温度センサ値	
24～25	CPU ファン回転状態	0～65535 回転数、符号無整数値
26～55	電源ファームウェアバージョン	ASCII、30BYTE、前詰め、終端に NULL(0x00) (詳細説明後述)
56～255	予約	未使用

電源ファームウェアバージョン

電源マイコンファームウェアのバージョンが ASCII 文字列で返されます。
(30BYTE 領域に前詰めで格納。文字列最後に NULL を設定。それ以降は不定。)

例:

バイト位置[26-55] バージョンが、"01.00.11" の場合

→ バイト位置 26 = '0' (0x30)、27 = '1' (0x31)、28 = '.' (0x3E)、29 = '0' (0x30)、
 30 = '0' (0x30)、31 = '.' (0x3E)、32 = '1' (0x31)、33 = '1' (0x31)、
 34 = '¥0' (0x00)、35~55 = 不定

.NET で使用できるデータに変換するには、以下のサンプルプログラムを参考にしてください。

サンプルプログラム:

```
Byte[] buffer = new Byte[256];    //モデル名格納用バッファー
string firm_version;              //string 形式のバージョン情報

                                // POS ハードウェア情報を取得する
if (GetPOSHwInfo(ref buffer[0]))
{
                                //バージョンを ASCII から string 形式に変換する
    firm_version = System.Text.Encoding.UTF8.GetString(buffer, 26, 30);
}
```

第9章 プロセス情報

9.1. プロセス情報取得 (ActiveX Control)

9.1.1. 概要

Windows 上で動作しているプロセスの情報を出力するための機能です。
プロセス情報を取得するための API (OCX) と、サンプルプログラムを提供します。

9.1.2. オブジェクト形態

オブジェクトタイプ	オブジェクトドライブリプション	プログラマティック ID
ActiveX Control	SHARP PROCESS INFORMATION CONTROL	SHARP.PROCLOG

9.1.3. 機能

プロセス情報取得コントロールは、Windows 上で動作しているプロセスの CPU 使用率やメモリ使用量などの情報をログファイルに保存することができます。

9.1.4. 共通 I/F サマリー

メソッド名称	ドライブリプション
ProcessLog	起動している全プロセスの情報をログファイルに出力します

9.1.5. メソッド詳細

ProcessLog メソッド

形式 [VC#] Int32 ProcessLog()
 [VB.NET] Function ProcessLog() As Int32

説明 起動している全プロセスの情報をログファイルに出力します。
戻り値 次の値のいずれかが戻されます。

値	意味
PFO_Success	正常終了
PFO_FileSpace_Error	書き込みスペースがない
PFO_FileProtect_Error	プロテクトエラー
PFO_FileWrite_Error	ライトエラー (システムエラー)
PFO_Memory_Error	メモリ関連エラー

9.1.6. ログファイル詳細

【概要】

プロセス情報オブジェクトが出力するログファイルです。

起動している全てのプロセスのプロセス ID、CPU 使用率、メモリ使用量、基本優先度等の情報を格納するためのファイルです。

【ログファイル形式】

ログファイルはデフォルトで次のファイル名、場所に保存されます。

出力先ファイル名 POSProc.log

出力先フォルダ名 %SystemRoot%\SysWOW64

※ 出力先ファイル名とフォルダは、設定ファイルの Name キーと Path キーにて指定する事が可能です。

※ ログ出力先のフォルダが存在しない場合、ログ出力されないためご注意ください。

【ログデータフォーマット】

プロセス情報は以下の形式で保存されます。なお、各項目は、‘,’ (セパレータ) で区切られます。

日(年/月/日)	時(時:分:秒.m 秒)	ログ識別子	ファイル名(文字列)	固定値(DEC)	(次行に続く)
YYYY/MM/DD	HH:MM:SS.sss	P	可変長	000000(6 桁固定)	

プロセス ID(HEX)	CPU 使用率(DEC:%)	メモリ使用量(DEC:バイト)	(次行に続く)
可変長	可変長	可変長(P:可変長, V:可変長)	

基本優先度(DEC)	補足情報(ウィンドウキャプション)	タミネータ
可変長	可変長	CR(0x0D)+LF(0x0A)

【ログファイル例】

```
2006/08/01,10:59:19.265,P,"Idle",000000,0x0,57,"20480(P:20480,V:0)",0,""  
2006/08/01,10:59:19.265,P,"System",000000,0x4,0,"245760(P:245760,V:0)",8,""  
2006/08/01,10:59:19.265,P,"smss.exe",000000,0x244,0,"651264(P:475136,V:176128)",11,""  
2006/08/01,10:59:19.265,P,"csrss.exe",000000,0x284,2,"5939200(P:4435968,V:1503232)",13,""  
2006/08/01,10:59:19.265,P,"winlogon.exe",000000,0x29C,0,"8065024(P:3870720,V:4194304)",13,"NetDDE Agent"  
2006/08/01,10:59:19.265,P,"services.exe",000000,0x2C8,1,"5296128(P:3436544,V:1859584)",9,""  
2006/08/01,10:59:19.265,P,"lsass.exe",000000,0x2D4,0,"4919296(P:1441792,V:3477504)",9,""  
2006/08/01,10:59:19.265,P,"svchost.exe",000000,0x360,0,"4968448(P:3551232,V:1417216)",8,""  
2006/08/01,10:59:19.265,P,"svchost.exe",000000,0x3EC,0,"24600576(P:14913536,V:9687040)",8,""  
2006/08/01,10:59:19.265,P,"svchost.exe",000000,0x4AC,0,"2658304(P:1966080,V:692224)",8,""  
2006/08/01,10:59:19.265,P,"svchost.exe",000000,0x4B8,0,"4730880(P:3624960,V:1105920)",8,""  
2006/08/01,10:59:19.265,P,"explorer",000000,0x5C8,2,"43110400(P:24690688,V:18419712)",8,"SYSTEM32"  
2006/08/01,10:59:19.265,P,"spoolsv.exe",000000,0x628,0,"5791744(P:4083712,V:1708032)",8,""  
2006/08/01,10:59:19.265,P,"ctfmon.exe",000000,0x664,0,"4726784(P:3174400,V:1552384)",8,""  
2006/08/01,10:59:19.265,P,"DUAgent.exe",000000,0x744,0,"1818624(P:1421312,V:397312)",8,""  
2006/08/01,10:59:19.265,P,"Pwfail.exe",000000,0x774,0,"2121728(P:1626112,V:495616)",8,""  
2006/08/01,10:59:19.265,P,"snmp.exe",000000,0x790,0,"4972544(P:3547136,V:1425408)",8,""  
2006/08/01,10:59:19.265,P,"inetinfo.exe",000000,0x3C8,0,"7680000(P:5795840,V:1884160)",8,""  
2006/08/01,10:59:19.265,P,"PFOTest.exe",000000,0x330,11,"17002496(P:11075584,V:5926912)",8,"Project1"  
2006/08/01,10:59:19.265,P,"PFOCALC.exe",000000,0x5E8,0,"3563520(P:2797568,V:765952)",8,""  
2006/08/01,10:59:19.265,P,"wmiprvse.exe",000000,0x650,0,"5599232(P:3710976,V:1888256)",8,""  
2006/08/01,10:59:19.265,P,"wmiprvse.exe",000000,0xA4,0,"4493312(P:3313664,V:1179648)",8,""  
2006/08/01,10:59:19.265,P,"Main_clr.exe",000000,0x6D0,14,"23805952(P:13750272,V:10055680)",8,"MAIN"  
2006/08/01,10:59:19.265,P,"Menu_clr",000000,0x728,8,"19185664(P:12558336,V:6627328)",8,"MENU"  
2006/08/01,10:59:19.265,P,"_Total",000000,0x0,100,"205967360(P:129527808,V:76439552)",0,""
```

9.1.7. 設定ファイル (POSProc.ini)

【概要】

プロセス情報オブジェクト用設定ファイルです。
ログ出力先、最大ファイルサイズ等の設定を行うためのファイルです。

【設定ファイル (POSProc ini) の格納場所】

PFOProc.ocx と共通のフォルダに格納されます。

ファイル名 POSProc.ini
格納先フォルダ名 %SystemRoot%\SysWOW64
※ 設定ファイルが存在しなかった場合は、デフォルトの設定で動作を行います。

【設定内容】

設定キー	内容	未指定時	例
LogMode	ログ出力モードの指定(※1) 0:メソッドのみ 1:メソッド及び定期出力	0(メソッドのみ)	LogMode =0
Path	ログ出力フォルダ指定	C:\Windows\SysWOW64	Path=C:\Apl\POSoplog
Name	ログ出力ファイル名指定	POSProc.log	Name=POSProc.log
Size	最大ファイルサイズ指定(※2) (単位:KByte)	0(無限)	Size=3000 (3MByte)
Interval	ログ出力間隔指定(※3) (単位:秒)	5(5 秒)	Interval=10 (10 秒)

※1 定期出力はアプリ開発段階の分析用として利用します。実運用では使用しません。

※2 ログは 1 行単位で出力され、書き込み前にファイルサイズの確認が行われる為、最大ファイルサイズを少し超えた状態でログ出力を停止します。

※3 ログの定期出力時の間隔を指定します。
これは同時にプロセス情報のサンプリング間隔も意味します。
Interval に 0 を指定することは出来ません。0 を指定すると 1 として扱います。

【設定ファイル例(出荷時設定)】

```
[PFOProc]

#ログ出力モード
LogMode = 0

#ログ出力フォルダ
Path = C:\Windows\system32

#ログ出力ファイル名
Name = POSProc.log

#最大ファイルサイズ(単位:KByte)
Size = 0

#ログ出力間隔(単位:秒)
Interval = 5
```

(注意) ・設定ファイルは Windows の一般的な設定ファイルの形式で記述します。

・Path 設定は「C:\Windows\system32」ですが、Windows10 の WOW64 システムにより、「C:\Windows\SysWOW64」フォルダに出力されます。

第10章 操作ログ

10.1. 操作ログ(ActiveX Control)

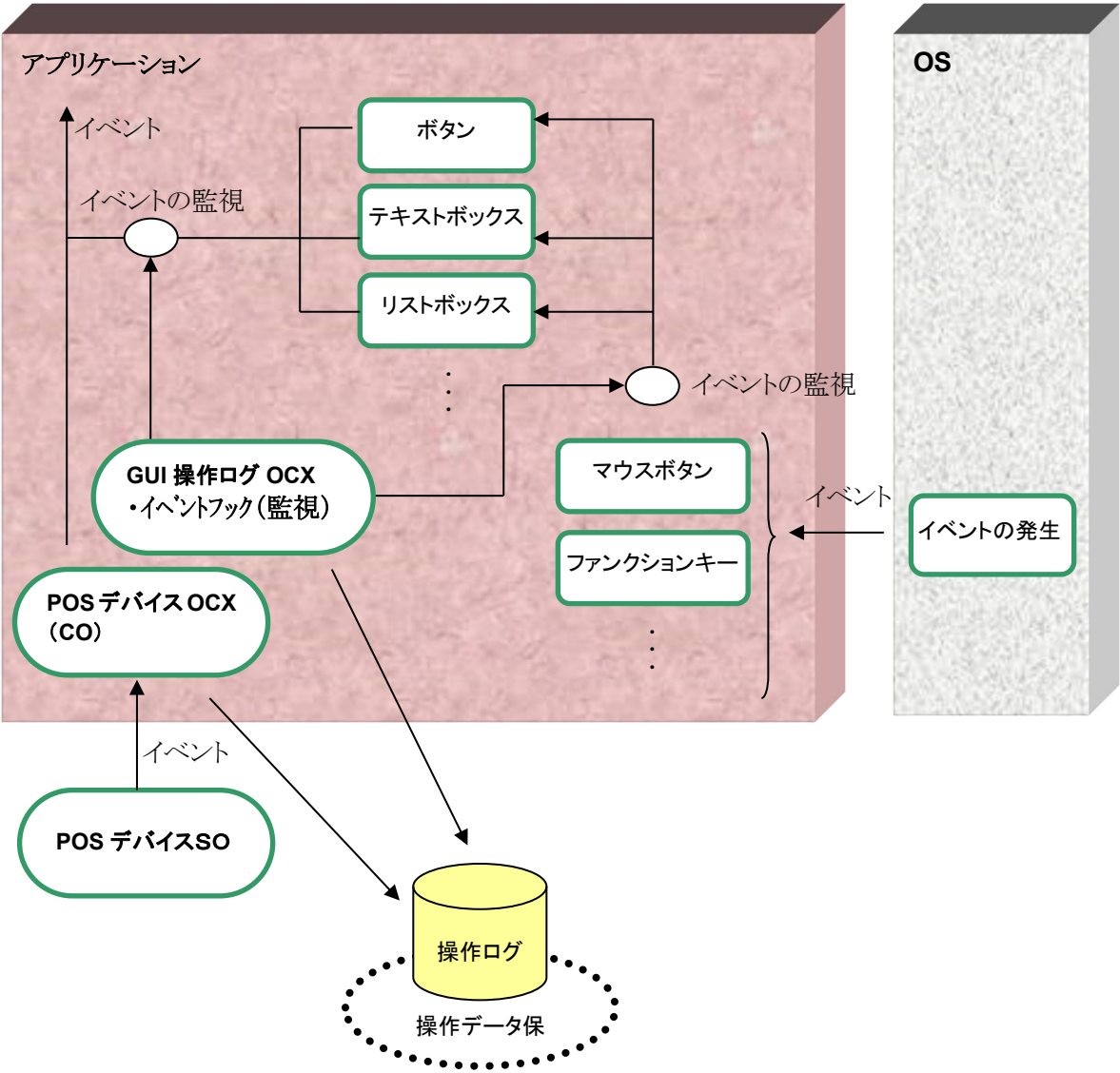
10.1.1. 概要

操作(POS キー、タッチパネル、スキャナ、MSR)情報をロギングします。
本ソフトウェアをアプリケーションに組み込むことで、アプリ実行時の操作内容をロギングします。

操作ログオブジェクト体系

分類	オブジェクト	機能
GUI	SHARP.POS.PFO. GUIOpeLog.dll	標準入力(POS キー、タッチパネル)のログ
POS デバイス	scanner.ocx	OPOS-API(CO)
	Msr.ocx	OPOS-API(CO)

操作ログ概念図



10.1.2. モジュール形態

オブジェクト	オブジェクトタイプ	オブジェクトドライプリプション	プログラマティック ID
SHARP.POS.PFO. GUIOpeLog.dll	.NET Class Library	-	-
scanner.ocx	ActiveX Control	OPOS Scanner OLE Control	OPOS.Scanner
Msr.ocx	ActiveX Control	OPOS Msr OLE Control	OPOS.MSR

10.1.3. 操作ログ対応モジュールの利用方法

分類	利用方法
GUI	フォーム毎に.NET コンポーネント GUIOpeLog を貼り付けることで、操作ログが可能となります。(プロパティ、メソッドの呼び出しは必要ありません) アプリ(1 プロセス)が複数フォームを所有している場合、それぞれのフォームに貼り付ける必要があります。
POS デバイス	OPOS-API(スキャナ/MSR OCX)をフォームに貼り付けます。 OPOS レジストリーに操作ログの指定を行います。 デバイス制御および入力量ログ出力が行われます。

10.1.4. GUI 操作ログ対象オブジェクト

名称	GUI, POS デバイス	オブジェクト名称	補足
GUIOpeLog	フォーム	Form	Visual Studio の Windows フォーム標準部品 (ツールバーに初期登録 されている部品)
	ラベル	Label	
	リンクラベル	LinkLabel	
	ボタン	Button	
	テキストボックス	TextBox	
	チェックボックス	CheckBox	
	ラジオボタン	RadioButton	
	ピクチャボックス	PictureBox	
	パネル	Panel	
	データグリッド	DataGrid	
	リストボックス	ListBox	
	チェックドリストボックス	CheckedListBox	
	コンボボックス	ComboBox	
	リストビュー	ListView	
	ツリービュー	TreeView	
	タブコントロール	TabControl (TabPage を含む)	
	DateTimePicker	DateTimePicker	
	月間カレンダー	MonthCalendar	
	水平スクロールバー	HScrollBar	
	垂直スクロールバー	VScrollBar	
	スプリッター	Splitter	
	ドメインアップダウン	DomainUpDown	
	ナメリックアップダウン	NumericUpDown	
	トラックバー	TrackBar	
	リッチテキストボックス	RichTextBox	
Scanner	スキャナ	Scanner	入力、エラーのイベントをロギング
Msr	MSR	Msr	入力(入力データは含まない)、書き込み、エラーのイベントをロギング

10.1.5. 設定ファイル (posoplog.ini)

【設定ファイル (posoplog.ini) の格納場所】

ファイル名 posoplog.ini

格納先フォルダ名 %SystemRoot%\SysWOW64

※ 設定ファイルが存在しなかった場合は、デフォルトの設定で動作を行います。

設定キー	内容	未指定時	例
Enable	ログ出力の有無指定(※3) 1:出力有 0:出力無	ログ出力有	例 Enable=0
Path	ログ出力フォルダ指定	c:\posoplog	例 Path=c:\apl\posoplog
name	ログ出力ファイル名指定	OP.log	例 Name=operation.log
size	最大ファイルサイズ指定(※1) (単位:KByte)	0(無限)	例 size=3000 (3MByte)
CRConvert	改行コード置換え文字列(※2)	(CR)	例 CRConvert=
CharacterCode	ログ出力ファイルの文字コード の指定(※2) UTF-16:UNICODE Shift_JIS:Shift JIS	Shift_JIS	例 CharacterCode=UTF-16

※1 ログは1行単位で出力され、書き込み前にファイルサイズの確認が行われるため、最大ファイルサイズを少し超えた状態でログ出力を停止します。

※2 本設定は、.NET 用オブジェクト(SHARP.POS.PFO.GUIOpeLog.dll)の標準入力操作ログにのみ有効です。

※3 本設定は、GUI 操作ログのみ有効です。デバイス操作ログはレジストリで切り替えます。詳しくは「RZ-A476S/A396S POS デバイス API 説明書」を参照ください。

【設定ファイル例(出荷時設定)】

```
[Log]
Enable=
Path=c:\posoplog
Name=oplog.log
Size=2000
CharacterCode= Shift_JIS
```

(注意) posoplog.ini ファイルの先頭には [Log] のラベルを記述してください。

10.1.6. ログデータフォーマット

以下の形式で保存されます。なお、各項目は、‘,’ (セパレータ) で区切られます。
ログファイルは、設定ファイルで指定されたフォルダが存在しない場合、出力されません。

項目 No.	項目名	形式	テキストボックスにテキストを入力した場合の一例
1	年/月/日	YYYY/MM/DD	2017/07/11
2	時(時:分:秒.m 秒)	HH:MM:SS.sss	10:10:25.906
3	ログ識別子	S	S
4	PG-ID (EXE ファイル名)	可変長	"posoplogtester"
5	固定値	000000 (6 桁固定)	000000
6	プロセスID	可変長	0x1a4
7	フォームウインドウキャプション	可変長	"uriage"
8	フォームウインドウハンドル	可変長	0x180886
9	操作情報	可変長	"TextBox(Text)"
10	補足(必要に応じて)	可変長	"X=8 Y=176"
11	タミネータ	CR(0x0D) + LF(0x0A)	0x0D, 0x0A

(補足)

- ・PG-ID (EXE ファイル名)、フォームウインドウキャプション、操作情報、補足は、ダブルクォーテーション「"」で括られ出力されます。
- ・プロセス ID、フォームウインドウハンドル、16 進表記 (0xXXXX) で出力されます。
その他の情報は、ASCII 文字列として出力されます。

例) テキストボックスにテキストを入力した場合

```
2017/07/11, 10:10:25.843, S, " posoplogtester" , 000000, 0x1a4, " uriage" , 0x180886, " TextBox-KeyDown" , " X=8 Y=176  
Key=0x54"  
2017/07/11, 10:10:25.906, S, " posoplogtester" , 000000, 0x1a4, " uriage" , 0x180886, " TextBox(Text)" , " X=8 Y=176"
```

例) マルチラインのテキストボックスで改行を入力した場合

改行コード置換え文字列設定
CRConvert=(CR)

```
2017/07/11, 15:30:06.671, S, " posoplogtester" , 000000, 0xf80, " uriage" , 0xc051c, " TextBox-KeyDown" , " X=8 Y=176  
Key=0xd"  
2017/07/11, 15:30:06.671, S, " posoplogtester" , 000000, 0xf80, " uriage" , 0xc051c, " TextBox(Text (CR) Box)" , " X=8  
Y=176"
```

10.1.7. 操作情報、補足フォーマット

.NET GUI

フォーム

コンポーネント名	Form (.NET Windows フォーム)	
事象	操作情報(オブジェクト内容)	補足情報
ロード	Form-Load	X=xxx Y=xxx Mouse=xxx xxx フォーム座標、マウス座標
アンロード	Form-Unload	
アクティブ	Form-Activated	
マウス入力	Form-Click	
キー入力	Form-KeyDown	X=xxx Y=xxx Key=xxx フォーム座標、キーコード

※ アクティブについては、デザイナーで最大化設定(WindowState プロパティを Maximized に設定)を行っていた場合、.NET の仕様により Load よりも Activated が先にロギングされる場合があります。

ラベル

コンポーネント名	Label (.NET Windows フォーム標準ラベル)	
事象	操作情報(オブジェクト内容)	補足情報
マウス入力	Label-Click (ラベル名)	X=xxx Y=xxx Mouse=xxx xxx ラベル座標、マウス座標

リンクラベル

コンポーネント名	LinkLabel (.NET Windows フォーム標準ラベル)	
事象	操作情報(オブジェクト内容)	補足情報
リンククリック	LinkLabel-LinkClick (リンクラベル名)	X=xxx Y=xxx リンクラベル座標

ボタン

コンポーネント名	Button (.NET Windows フォーム標準ボタン)	
事象	操作情報(オブジェクト内容)	補足情報
ボタン押下	Button-Click (ボタンラベル名)	X=xxx Y=xxx ボタン座標
キー入力	Button-KeyUp	X=xxx Y=xxx Key=xxx ボタン座標、キーコード

テキストボックス

コンポーネント名	TextBox (.NET Windows フォーム標準テキストボックス)	
事象	操作情報(オブジェクト内容)	補足情報
テキスト変更	TextBox (テキスト内容全体)	X=xxx Y=xxx テキストボックス座標
キー入力	TextBox-KeyDown	X=xxx Y=xxx Key=xxx テキストボックス座標、キーコード

チェックボックス

コンポーネント名	CheckBox (.NET Windows フォーム標準チェックボックス)	
事象	操作情報(オブジェクト内容)	補足情報
オン	CheckBox-ON (チェックボックスラベル名)	X=xxx Y=xxx チェックボックス座標
オフ	CheckBox-OFF (チェックボックスラベル名)	X=xxx Y=xxx チェックボックス座標
キー入力	CheckBox-KeyUp (チェックボックスラベル名)	X=xxx Y=xxx Key=xxx チェックボックス座標、キーコード

ラジオボタン

コンポーネント名	RadioButton (.NET Windows フォーム標準ラジオボタン)	
事象	操作情報(オブジェクト内容)	補足情報
オン	RadioButton-ON (ラジオボタンラベル名)	X=xxx Y=xxx ラジオボタン座標
キー入力	RadioButton-KeyUp (ラジオボタンラベル名)	X=xxx Y=xxx Key=xxx ラジオボタン座標、キーコード

ピクチャボックス

コンポーネント名	PictureBox (.NET Windows フォーム標準ピクチャボックス)	
事象	操作情報(オブジェクト内容)	補足情報
マウス入力	PictureBox-Click	X=xxx Y=xxx Mouse=xxx xxx ピクチャボックス座標、マウス座標

パネル

コンポーネント名	Panel (.NET Windows フォーム標準パネル)	
事象	操作情報(オブジェクト内容)	補足情報
マウス入力	Panel-Click	X=xxx Y=xxx Mouse=xxx xxx パネル座標、マウス座標

データグリッド

コンポーネント名	DataGrid(.NET Windows フォーム標準データグリッド)	
事象	操作情報(オブジェクト内容)	補足情報
選択セル変更	DataGrid-Cell(X,Y)	X=xxx Y=xxx データグリッド座標

データグリッドテキストボックス

コンポーネント名	DataGridTextBox(DataGrid 内テキストボックス)	
事象	操作情報(オブジェクト内容)	補足情報
テキスト変更	DataGridTextBox (テキスト内容全体)	X=xxx Y=xxx データグリッドテキストボックス座標
キー入力	DataGridTextBox-KeyDown	X=xxx Y=xxx Key=xxx データグリッドテキストボックス座標、 キーコード

※ コンポーネント特性によりデータグリッドの選択セル変更を行った際、テキスト変更ログが出力されます。

リストボックス

コンポーネント名	ListBox (.NET Windows フォーム標準リストボックス)	
事象	操作情報(オブジェクト内容)	補足情報
選択	ListBox-Select (選択された項目の文字列)	X=xxx Y=xxx リストボックス座標
マウス入力	ListBox-Click	X=xxx Y=xxx Mouse=xxx xxx リストボックス座標、マウス座標
キー入力	ListBox-KeyDown	X=xxx Y=xxx Key=xxx リストボックス座標、キーコード

※ シングル選択のみ対応。

チェックドリストボックス

コンポーネント名	CheckedListBox (.NET Windows フォーム標準チェックドリストボックス)	
事象	操作情報(オブジェクト内容)	補足情報
選択	CheckedListBox-Select (選択された項目の文字列)	X=xxx Y=xxx チェックドリストボックス座標
オン	CheckedListBox-ON (ON にした項目の文字列)	X=xxx Y=xxx チェックドリストボックス座標
オフ	CheckedListBox-OFF (OFF にした項目の文字列)	X=xxx Y=xxx チェックドリストボックス座標

コンボボックス

コンポーネント名	ComboBox (.NET Windows フォーム標準コンボボックス)	
事象	操作情報(オブジェクト内容)	補足情報
選択 (テキスト変更含む)	ComboBox-Select (テキスト内容全体)	X=xxx Y=xxx コンボボックス座標
キー入力	ComboBox-KeyDown	X=xxx Y=xxx Key=xxx コンボボックス座標、キーコード

リストビュー

コンポーネント名	ListView (.NET Windows フォーム標準リストビュー)	
事象	操作情報(オブジェクト内容)	補足情報
選択	ListView-Select (選択された項目のメインアイテム文字列)	X=xxx Y=xxx リストビュー座標
オン	ListView-ON (ON にした項目のメインアイテム文字列)	X=xxx Y=xxx リストビュー座標
オフ	ListView-OFF (OFF にした項目のメインアイテム文字列)	X=xxx Y=xxx リストビュー座標
ラベル変更	ListView-LabelChange	X=xxx Y=xxx ItemIndex=xxx リストビュー座標、 ラベル変更アイテムインデックス

※ シングル選択のみ対応。ラベル変更は変更後のラベルを正しく判別できません。
コンポーネント特性により選択アイテムを変更した場合、選択解除／選択のログが出力されます。

ツリービュー

コンポーネント名	TreeView (.NET Windows フォーム標準ツリービュー)	
事象	操作情報(オブジェクト内容)	補足情報
選択	TreeView-Select (選択された項目の文字列)	X=xxx Y=xxx Action=Key/Mouse ツリービュー座標、入力デバイス
オン	TreeView-ON (ON にした項目のアイテム文字列)	X=xxx Y=xxx Action=Key/Mouse ツリービュー座標、入力デバイス
オフ	TreeView-OFF (OFF にした項目の文字列)	X=xxx Y=xxx Action=Key/Mouse ツリービュー座標、入力デバイス
ラベル変更	TreeView-LabelChange	X=xxx Y=xxx ツリービュー座標

※ ラベル変更は変更後のラベルを正しく判別できません。

タブコントロール

コンポーネント名	TabControl (.NET Windows フォームタブコントロール)	
事象	操作情報(オブジェクト内容)	補足情報
選択	TabControl-Select (選択された項目の文字列)	X=xxx Y=xxx タブコントロール座標
キー入力	TabControl-KeyDown	X=xxx Y=xxx Key=xxx タブコントロール座標、キーコード

タブページ

コンポーネント名	TabPage (TabControl 内のタブページコントロール)	
事象	操作情報(オブジェクト内容)	補足情報
マウス入力	TabPage-Click	X=xxx Y=xxx Mouse=xxx xxx タブページコントロール座標、 マウス座標

データタイムピッカー

コンポーネント名	DateTimePicker (.NET Windows フォームデータタイムピッカー)	
事象	操作情報(オブジェクト内容)	補足情報
日時変更	DateTimePicker (表示されている日時文字列)	X=xxx Y=xxx データタイムピッカー座標
マウス入力	DateTimePicker-Click	X=xxx Y=xxx Mouse=xxx xxx データタイムピッカー座標、 マウス座標
キー入力	DateTimePicker-KeyDown	X=xxx Y=xxx Key=xxx データタイムピッカー座標、キーコード

※ ドロップダウンのマウス入力はロギングされません。

オペレーションにより MouseUp/Down の通知(ロギング)がされる場合とされない場合があります。

月間カレンダー

コンポーネント名	MonthCalendar (.NET Windows フォーム月間カレンダー)	
事象	操作情報(オブジェクト内容)	補足情報
マウス入力	MonthCalendar-Click	X=xxx Y=xxx Mouse=xxx xxx 月間カレンダー座標、マウス座標
キー入力	MonthCalendar-KeyDown	X=xxx Y=xxx Key=xxx 月間カレンダー座標、キーコード

※ 上部日付クリックによるアップダウンコントロール内での操作、及びポップアップウィンドウでの入力はロギングされません。

水平スクロールバー

コンポーネント名	HScrollBar (.NET Windows フォーム水平スクロールバー)	
事象	操作情報(オブジェクト内容)	補足情報
スクロール (ValueChanged)	HScrollBar.Scroll	X=xxx Y=xxx ScrollMin=xxx ScrollMax=xxx BarPosition=xxx 水平スクロールバー座標、 スクロール最小値、 スクロール最大値、スクロール位置

垂直スクロールバー

コンポーネント名	VScrollBar (.NET Windows フォーム垂直スクロールバー)	
事象	操作情報(オブジェクト内容)	補足情報
スクロール (ValueChanged)	VScrollBar.Scroll	X=xxx Y=xxx ScrollMin=xxx ScrollMax=xxx BarPosition=xxx 垂直スクロールバー座標、 スクロール最小値、 スクロール最大値、スクロール位置

スプリッター

コンポーネント名	Splitter (.NET Windows フォームスプリッター)	
事象	操作情報(オブジェクト内容)	補足情報
分割移動	Splitter.Moved	X=xxx Y=xxx スプリッター座標

ドメインアップダウン

コンポーネント名	DomainUpDown (.NET Windows フォームドメインアップダウン)	
事象	操作情報(オブジェクト内容)	補足情報
選択	DomainUpDown.Select (テキスト内容全体)	X=xxx Y=xxx ドメインアップダウン座標
キー入力	DomainUpDown.KeyDown	X=xxx Y=xxx Key=xxx ドメインアップダウン座標、キーコード

ナメリックアップダウン

コンポーネント名	NumericUpDown (.NET Windows フォームナメリックアップダウン)	
事象	操作情報(オブジェクト内容)	補足情報
選択	NumericUpDown (選択した値の文字列)	X=xxx Y=xxx ナメリックアップダウン座標
キー入力	NumericUpDown.KeyDown	X=xxx Y=xxx Key=xxx ナメリックアップダウン座標、 キーコード

トラックバー

コンポーネント名	TrackBar (.NET Windows フォームトラックバー)	
事象	操作情報(オブジェクト内容)	補足情報
選択	TrackBar(選択した値)	X=xxx Y=xxx トラックバー座標

リッチテキストボックス

コンポーネント名 **RichTextBox (.NET Windows フォームリッチテキストボックス)**

事象	操作情報 (オブジェクト内容)	補足情報
テキスト変更	RichTextBox (テキスト内容全体)	X=xxx Y=xxx リッチテキストボックス座標
キー入力	RichTextBox-KeyDown	X=xxx Y=xxx Key=xxx リッチテキストボックス座標、 キーコード

POS デバイス

スキャナ

コンポーネント名	Scanner (スキャナーCO)	
事象	操作情報(オブジェクト内容)	補足情報
読み取り通知	ScannerDataEvent (ScanData)	
エラー通知	ScannerErrorEvent (Resultcode, ResultCodeExtended)	

※ ScanData、Resultcode、ResultCodeExtended には、OPOS のプロパティ値がセットされます。

MSR

コンポーネント名	MSR (MSR-CO)	
事象	操作情報(オブジェクト内容)	補足情報
読み取り通知	MSRDataEvent ()	
書き込み通知	MSRWriteCompleteEvent ()	
エラー通知	<u>読み取りエラー時</u> MSRErrorEvent (Read:Resultcode, ResultCodeExtended) <u>書き込みエラー時</u> MSRErrorEvent (Write:Resultcode, ResultCodeExtended)	

※ Resultcode、ResultCodeExtended には、OPOS のプロパティ値がセットされます。

10.1.8. 操作ログ制約・注意事項

- 本操作ログソフトウェアは、Visual Studio 2015 アプリケーションから利用することを前提に設計しています。他の開発言語やサードベンダー製 GUI 部品についてはサポートしておりません。
- 操作ログは、外部操作(キー:Keydown/KeyUp, マウス:Click)とフォーム(Load/Active/Unload)、POS デバイス入力に関する内容がロギングされます。
- 仮想入力(PerformClick 等)もロギングされます。
- プログラムにより行ったデータ操作のイベントもロギングされます。
- GUI 操作ログオブジェクトは、フォーム毎に貼り付ける必要があります。メッセージボックスに関してはログを取る事が出来ません。
- GUI 操作ログは、イベントハンドラ登録順にて実行されるため基本的にアプリケーションイベントハンドラ実行後にロギングされます。
- GUI 操作ログはログ書き込み先ドライブの空き領域が 1M バイト以下となった場合、ロギングを停止します。停止時には補足情報に次の文字列を出力しロギングを停止します。
「"GUIOpenLog : Log is stopped.(Free disk is less than 1MB.)"」
- POS デバイス(MSR、SCANNER) 操作ログはログ書き込み先ドライブの空き領域が 1M バイト以下となった場合、ロギングを停止します。停止時には補足情報に次の文字列を出力しロギングを停止します。停止時には補足情報に次の文字列をそれぞれ出力しロギングを停止します。
「"(デバイスクラス) : Log is stopped.(Free disk is less than 1MB.)"」
- デバイス(MSR、SCANNER) ログは、ハードウェアなしエラーが連続でログ出力されるのを制限しています。ハードウェアなしエラーが連続で 4 回発生すると次のような文字列を補足情報に出力し、ロギングを一時停止します。
「"MSRErrorEvent is temporarily interrupted."」
「"ScannerErrorEvent is temporarily interrupted."」
別のイベントが発生するか、4 回目のエラーから 1 分以上経過していた場合、ロギングを再開します。
- MSR における読み取りデータはロギングしていません。
- 監視対象外のオブジェクトに関しては、操作ログ情報が残る可能性がありますが、内容については保証されません。

付録A. PFO 定数一覧

PFO にて定義している定数一覧です。

これらの定数はオブジェクト参照設定を行うことで利用することができます。

定数	値	意味
PFO_DL_BacklightOn	1	バックライト On
PFO_DL_BacklightOff	0	バックライト Off
PFO_TV_ScrollUp	0	スクロールアップ
PFO_TV_ScrollDown	1	スクロールダウン
PFO_TV_ScrollLeft	2	スクロールレフト
PFO_TV_ScrollRight	3	スクロールライト
PFO_EJ_WriteMode	1	ライトモード
PFO_EJ_WriteBufferMode	2	ライトモード(バッファ利用モード)
PFO_EJ_ReadMode	3	リードモード
PFO_EJ_CreateNormal	0	標準モード
PFO_EJ_CreateHidden	1	隠し属性モード
PFO_EJ_DeleteOld	0	更新日付の最も古いファイル削除
PFO_EJ_DeleteSel	1	指定ファイル削除
PFO_EJ_FindFirst	0	最初のファイル検索
PFO_EJ_FindNext	1	次のファイル検索
PFO_EJ_UIMode_Disable	1	UI 禁止
PFO_EJ_UIMode_Enable	2	UI 許可(スクロールバーオフ)
PFO_EJ_UIMode_EnableS	3	UI 許可(スクロールバーオン)
PFO_EJ_BackTrans	0	背景透過
PFO_EJ_BackColor	1	BackColor で背景描画
PFO_EJ_BackImage	2	BackImage で背景描画
PFO_JV_Idle	1	非ビジー(非印刷中)状態です
PFO_JV_Busy	2	ビジー(印刷中)状態です
PFO_JV_PrintStart	1	印刷ボタンが押されました
PFO_JV_PrintEnd	2	印刷が終了または中断されました
PFO_JV_NoCut	0	カットとロゴイメージ(スタンプ)印刷は行わない
PFO_JV_Cut	1	カットのみ行う
PFO_JV_CutStamp	2	カットとロゴイメージ(スタンプ)印刷を行う

定数	値	意味
PFO_Success	0	正常終了
PFO_Illegal_Error	-1	パラメーター不正 (ファイル名, ファイルハンドル, 有効範囲外, 無効データ)
PFO_Exist_Error	-2	既にファイルが存在します
PFO_NoExist_Error	-3	ファイルが存在しません
PFO_FileFormat_Error	-4	ファイルフォーマット不正
PFO_FileSpace_Error	-5	書き込みスペースがない
PFO_FileProtect_Error	-6	プロテクトエラー (ハードプロテクトまたはリードオンリー属性)
PFO_FileWrite_Error	-7	ライトエラー (システムエラー) - ハードエラー, PCカード寿命エラーなど
PFO_FileRead_Error	-8	リードエラー (システムエラー) - ハードエラーなど
PFO_Device_Error	-9	デバイス (またはメディア) が存在しない
PFO_DeviceName_Error	-10	デバイス名が存在しません。
PFO_Memory_Error	-11	メモリリソース関連エラー

付録B. Calcudgt 計算規則(方法)

(データ) (重み)

(2010年 34、33、32、31 (Hex)

(2010年 101(10-1)指定の場合

8 7 1 1 データ(38,37,31,31,30[h])各バイトについて、上位4バイトを無視して処理

× × × ×

52 51 50 49 重みは、1桁ずつ10進に直して処理

416 357 50 49 416と357はそれぞれ4と1と6、3と5と7に分ける

4+1+6+3+5+7+5+0+4+9=44

44÷10= 4 余り 4

10-4= 6 チェックデジット 6

(2010年 102(10-2)指定の場合

上記例では

416+357+50+49=872

872÷10= 87 余り 2

10-2= 8 チェックデジット 8

(2010年 111~115(11-1~11-5)指定の場合

上記例では

416+357+50+49=872

872÷11= 79 余り 3

11-3= 8 チェックデジット 8

(2010年 10×指定で、余りが0の場合

チェックデジット 0

(2010年 11×指定で、余りが0、1の場合

チェックデジット	余りが0の時	余りが1の時
111(11-1の場合)	無効(-2)	0
112(11-2の場合)	1	0
113(11-3の場合)	0	0
114(11-4の場合)	0	1
115(11-5の場合)	1	1

付録C. 演算データ形式

PS タイプ

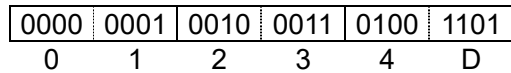


1 バイト～8 バイト

符号:B:正、D:負

値の範囲:1 桁～15 桁(正負共に)

(例) -1234



NS タイプ

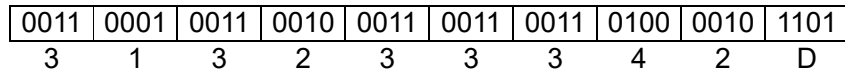


1 バイト～15 バイト

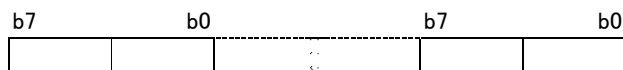
符号:2B:正、2D:負

値の範囲:1 桁～14 桁(正負共に)

(例) -1234



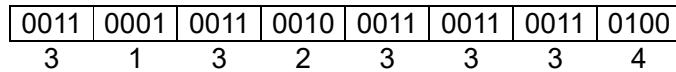
N タイプ



1 バイト～15 バイト

値の範囲:1 桁～15 桁

(例) 1234



付録D. 演算変換テーブル

JIS8→JIS7 コード変換テーブル

	00	10	20	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0
00	00h	10h	20h	30h	40h	50h	60h	70h	00h	00h	00h	30h	40h	50h	00h	00h
01	01h	11h	21h	31h	41h	51h	61h	71h	00h	00h	21h	31h	41h	51h	00h	00h
02	02h	12h	22h	32h	42h	52h	62h	72h	00h	00h	22h	32h	42h	52h	00h	00h
03	03h	13h	23h	33h	43h	53h	63h	73h	00h	00h	23h	33h	43h	53h	00h	00h
04	04h	14h	24h	34h	44h	54h	64h	74h	00h	00h	24h	34h	44h	54h	00h	00h
05	05h	15h	25h	35h	45h	55h	65h	75h	00h	00h	25h	35h	45h	55h	00h	00h
06	06h	16h	26h	36h	46h	56h	66h	76h	00h	00h	26h	36h	46h	56h	00h	00h
07	07h	17h	27h	37h	47h	57h	67h	77h	00h	00h	27h	37h	47h	57h	00h	00h
08	08h	18h	28h	38h	48h	58h	68h	78h	00h	00h	28h	38h	48h	58h	00h	00h
09	09h	19h	29h	39h	49h	59h	69h	79h	00h	00h	29h	39h	49h	59h	00h	00h
0A	0Ah	1Ah	2Ah	3Ah	4Ah	5Ah	6Ah	7Ah	00h	00h	2Ah	3Ah	4Ah	5Ah	00h	00h
0B	0Bh	1Bh	2Bh	3Bh	4Bh	5Bh	6Bh	7Bh	00h	00h	2Bh	3Bh	4Bh	5Bh	00h	00h
0C	0Ch	1Ch	2Ch	3Ch	4Ch	5Ch	6Ch	7Ch	00h	00h	2Ch	3Ch	4Ch	5Ch	00h	00h
0D	0Dh	1Dh	2Dh	3Dh	4Dh	5Dh	6Dh	7Dh	00h	00h	2Dh	3Dh	4Dh	5Dh	00h	00h
0E	0Eh	1Eh	2Eh	3Eh	4Eh	5Eh	6Eh	7Eh	00h	00h	2Eh	3Eh	4Eh	5Eh	00h	00h
0F	0Fh	1Fh	2Fh	3Fh	4Fh	5Fh	6Fh	7Fh	00h	00h	2Fh	3Fh	4Fh	5Fh	00h	00h

(2010年 シフトインモードからシフトアウトモードに移行する場合 SO(0Eh)が付加されます。

シフトアウトモードからシフトインモードに移行する場合 SI(0Fh)が付加されます。

初期状態はシフトインモードです。

JIS7→JIS8 コード変換テーブル

	シフトイン								シフトアウト							
	00	10	20	30	40	50	60	70	00	10	20	30	40	50	60	70
00	00h	10h	20h	30h	40h	50h	60h	70h	00h	10h	20h	B0h	C0h	D0h	00h	00h
01	01h	11h	21h	31h	41h	51h	61h	71h	01h	11h	A1h	B1h	C1h	D1h	00h	00h
02	02h	12h	22h	32h	42h	52h	62h	72h	02h	12h	A2h	B2h	C2h	D2h	00h	00h
03	03h	13h	23h	33h	43h	53h	63h	73h	03h	13h	A3h	B3h	C3h	D3h	00h	00h
04	04h	14h	24h	34h	44h	54h	64h	74h	04h	14h	A4h	B4h	C4h	D4h	00h	00h
05	05h	15h	25h	35h	45h	55h	65h	75h	05h	15h	A5h	B5h	C5h	D5h	00h	00h
06	06h	16h	26h	36h	46h	56h	66h	76h	06h	16h	A6h	B6h	C6h	D6h	00h	00h
07	07h	17h	27h	37h	47h	57h	67h	77h	07h	17h	A7h	B7h	C7h	D7h	00h	00h
08	08h	18h	28h	38h	48h	58h	68h	78h	08h	18h	A8h	B8h	C8h	D8h	00h	00h
09	09h	19h	29h	39h	49h	59h	69h	79h	09h	19h	A9h	B9h	C9h	D9h	00h	00h
0A	0Ah	1Ah	2Ah	3Ah	4Ah	5Ah	6Ah	7Ah	0Ah	1Ah	AAh	BAh	CAh	DAh	00h	00h
0B	0Bh	1Bh	2Bh	3Bh	4Bh	5Bh	6Bh	7Bh	0Bh	1Bh	ABh	BBh	CBh	DBh	00h	00h
0C	0Ch	1Ch	2Ch	3Ch	4Ch	5Ch	6Ch	7Ch	0Ch	1Ch	ACH	BCh	CCh	DCh	00h	00h
0D	0Dh	1Dh	2Dh	3Dh	4Dh	5Dh	6Dh	7Dh	0Dh	1Dh	ADh	BDh	CDh	DDh	00h	00h
0E	0Eh	1Eh	2Eh	3Eh	4Eh	5Eh	6Eh	7Eh	0Eh	1Eh	AEnh	BEh	CEh	DEh	00h	00h
0F	0Fh	1Fh	2Fh	3Fh	4Fh	5Fh	6Fh	7Fh	0Fh	1Fh	AFh	BFh	CFh	DFh	00h	7Fh

EBCDIC→JIS8 コード変換テーブル

	00	10	20	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0
00	00h	10h	00h	00h	20h	26h	2Dh	00h	00h	BFh	00h	00h	00h	00h	24h	30h
01	01h	00h	00h	00h	A1h	AAh	2Fh	00h	B1h	C0h	00h	00h	41h	4Ah	00h	31h
02	02h	00h	00h	16h	A2h	ABh	00h	00h	B2h	C1h	CDh	00h	42h	4Bh	53h	32h
03	03h	00h	00h	00h	A3h	ACH	00h	00h	B3h	C2h	CEh	00h	43h	4Ch	54h	33h
04	00h	00h	00h	00h	A4h	ADh	00h	00h	B4h	C3h	CFh	00h	44h	4Dh	55h	34h
05	09h	0Ah	0Ah	00h	A5h	Aeh	00h	00h	B5h	C4h	D0h	00h	45h	4Eh	56h	35h
06	00h	00h	17h	00h	A6h	AFh	00h	00h	B6h	C5h	D1h	00h	46h	4Fh	57h	36h
07	7Fh	00h	1Bh	04h	A7h	00h	00h	00h	B7h	C6h	D2h	00h	47h	50h	58h	37h
08	00h	00h	00h	00h	A8h	B0h	00h	00h	B8h	C7h	D3h	00h	48h	51h	59h	38h
09	00h	00h	00h	00h	A9h	00h	00h	00h	B9h	C8h	D4h	00h	49h	52h	5Ah	39h
0A	00h	00h	00h	00h	5Bh	5Dh	00h	3Ah	BAh	C9h	D5h	DAh	00h	00h	00h	00h
0B	00h	00h	00h	00h	2Eh	5Ch	2Ch	23h	00h	00h	00h	DBh	00h	00h	00h	00h
0C	00h	00h	00h	00h	3Ch	2Ah	25h	40h	BBh	00h	D6h	DCh	00h	00h	00h	00h
0D	00h	00h	05h	15h	28h	29h	5Fh	27h	BCh	CAh	D7h	DDh	00h	00h	00h	00h
0E	00h	00h	00h	00h	2Bh	3Bh	3Eh	3Dh	BDh	CBh	D8h	DEh	00h	00h	00h	00h
0F	00h	00h	00h	00h	21h	5Eh	3Fh	22h	BEh	CCh	D9h	DFh	00h	00h	00h	00h

JIS8→EBCDIC コード変換テーブル

	00	10	20	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0
00	00h	10h	40h	F0h	7Ch	D7h	00h	00h	00h	00h	00h	58h	91h	A5h	00h	00h
01	01h	00h	4Fh	F1h	C1h	D8h	00h	00h	00h	00h	41h	81h	92h	A6h	00h	00h
02	02h	00h	7Fh	F2h	C2h	D9h	00h	00h	00h	00h	42h	82h	93h	A7h	00h	00h
03	03h	00h	7Bh	F3h	C3h	E2h	00h	00h	00h	00h	43h	83h	94h	A8h	00h	00h
04	37h	00h	E0h	F4h	C4h	E3h	00h	00h	00h	00h	44h	84h	95h	A9h	00h	00h
05	2Dh	3Dh	6Ch	F5h	C5h	E4h	00h	00h	00h	00h	45h	85h	96h	AAh	00h	00h
06	00h	32h	50h	F6h	C6h	E5h	00h	00h	00h	00h	46h	86h	97h	ACH	00h	00h
07	00h	26h	7Dh	F7h	C7h	E6h	00h	00h	00h	00h	47h	87h	98h	ADh	00h	00h
08	00h	00h	4Dh	F8h	C8h	E7h	00h	00h	00h	00h	48h	88h	99h	Aeh	00h	00h
09	05h	00h	5Dh	F9h	C9h	E8h	00h	00h	00h	00h	49h	89h	9Ah	Afh	00h	00h
0A	15h	00h	5Ch	7Ah	D1h	E9h	00h	00h	00h	00h	51h	8Ah	9Dh	BAh	00h	00h
0B	00h	27h	4Eh	5Eh	D2h	4Ah	00h	00h	00h	00h	52h	8Ch	9Eh	BBh	00h	00h
0C	00h	00h	6Bh	4Ch	D3h	5Bh	00h	00h	00h	00h	53h	8Dh	9Fh	BCh	00h	00h
0D	00h	00h	60h	7Eh	D4h	5Ah	00h	00h	00h	00h	54h	8Eh	A2h	BDh	00h	00h
0E	00h	00h	4Bh	6Eh	D5h	5Fh	00h	00h	00h	00h	55h	8Fh	A3h	BEh	00h	00h
0F	00h	00h	61h	6Fh	D6h	6Dh	00h	07h	00h	00h	56h	90h	A4h	Bfh	00h	00h

付録E. ブザー制御

ブザー制御は、Windows 標準でサポートされる **Beep** 関数を使用します。

Beep 関数

形式 **[VC#]** `public static void Beep`
 `(int frequency, int duration)`
 ※`Console.Beep` メソッド

[VB.NET] `Public Shared Sub Beep`
 `( frequency As Integer, duration As Integer )`
 ※`Console.Beep` メソッド

パラメータ	説明
<i>frequency</i>	ビーブ音の周波数 37 ~ 32767 (単位:Hz)
<i>duration</i>	ビーブ音の再生時間 0 ~ 2147483647 (単位:ミリ秒)

説明 標準のシステムビーブ音が鳴ります。

【補足説明】

この関数は同期で実行されます。音が鳴り終わるまでは、呼び出し側へ制御を返しません。